

THE TOOLKIT · JGONZALEZ.APP

El Problema del Framing

Cómo decidir qué importa cuando todo cambia
— Un toolkit de framing para datos, IA y negocio

```
SELECT disciplina, criterio FROM toolkit WHERE supuesto =  
'explícito';
```

Jesús Mario González Siller

Data, Analytics & AI Solutions Architect

DATOS · ANALÍTICA · INTELIGENCIA ARTIFICIAL

El Problema del Framing

El Problema del Framing

Cómo decidir qué importa cuando todo cambia — Un toolkit de framing para datos, IA y negocio

© Jesús Mario González Siller. Todos los derechos reservados.

Primera edición.

Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida de ninguna forma ni por ningún medio, incluyendo fotocopiado, grabación u otros métodos electrónicos o mecánicos, sin el permiso previo por escrito del autor, excepto en el caso de citas breves incorporadas en reseñas críticas y ciertos otros usos no comerciales permitidos por la ley de derechos de autor.

Este libro presenta un framework conceptual de autoría propia, inspirado y con reconocimiento explícito al trabajo académico fundacional sobre el frame problem en inteligencia artificial (McCarthy & Hayes, 1969, y la literatura posterior citada en la bibliografía en formato APA). Los casos de negocio presentados son compuestos, anonimizados e ilustrativos; cualquier parecido con una organización real es coincidencia o generalización de patrones de industria.

Contacto y recursos complementarios: jgonzalez.app

*Para quienes alguna vez presentaron un plan
perfecto que el mundo real desarmó en una
semana — y decidieron entender por qué, en vez de
solo intentarlo con más fuerza la próxima vez.*

Lo que dicen los lectores

“Una forma de pensar que debería enseñarse en cualquier equipo de datos antes que cualquier herramienta técnica.”

— Líder de Analítica, retail multinacional

“Le puse nombre a un problema que llevaba años sintiendo sin poder explicar en juntas directivas.”

— Product Owner de IA, servicios financieros

“El Framework Canvas se volvió requisito de entrada en nuestro comité de proyectos de datos.”

— Director de Transformación Digital

Prólogo

En 1969, dos investigadores de inteligencia artificial —John McCarthy y Patrick Hayes— tropezaron con una pregunta que parecía trivial y resultó ser una de las más resistentes en la historia de su disciplina: ¿cómo le enseñas a un sistema a saber, de forma eficiente, qué se mantiene igual cuando algo en el mundo cambia? Le llamaron the frame problem (McCarthy & Hayes, 1969). Durante más de un cuarto de siglo, ocupó a algunos de los investigadores más brillantes de la lógica y la inteligencia artificial, generó debates públicos entre colegas, produjo el célebre y algo macabro Yale Shooting Problem (Hanks & McDermott, 1986), y dejó, al final del camino, no una solución única y definitiva, sino un catálogo extraordinariamente útil de estrategias parciales, cuidadosamente documentado por Morgenstern (1996) en el volumen que sirve de base principal a este libro.

Llevo años trabajando en la intersección de datos, analítica e inteligencia artificial aplicada a negocios de retail y consumo, y una y otra vez me he encontrado explicando, con otras palabras, exactamente el mismo problema: por qué un modelo que funcionaba perfecto en el papel falla en producción; por qué dos áreas de la misma empresa, mirando los mismos datos, llegan a conclusiones opuestas; por qué un plan estratégico impecable no sobrevive el primer trimestre de ejecución real. La causa, casi siempre, no es falta de talento ni falta de datos. Es falta de framing explícito: nadie hizo visible, a tiempo, qué se estaba asumiendo que seguiría igual.

Este libro nació de esa observación repetida. No es un libro de inteligencia artificial académica en sentido estricto, aunque toma prestadas sus mejores ideas y las cita con el rigor que merecen. Es un manual práctico, pensado para líderes de datos, producto e IA que necesitan tomar decisiones sólidas en entornos que no dejan de moverse. Cada capítulo cierra con un ejemplo extendido —compuesto y

anonimizado— y un ejercicio que puedes aplicar esta misma semana. Mi invitación no es que memorices la teoría. Es que la conviertas en hábito, entendiendo con precisión de dónde viene cada idea.

Cómo usar este libro

- Cada uno de los 30 capítulos cierra con un ejemplo real (compuesto y anonimizado) y un ejercicio aplicable de inmediato.
- Después de cada capítulo encontrarás una página de notas para conectar la idea con un proyecto propio — úsala, no la saltes.
- El libro está diseñado para leerse en orden la primera vez, y como referencia rápida después: cada capítulo es autocontenido.
- El Apéndice A contiene las fichas de trabajo del Toolkit listas para fotocopiar, imprimir o adaptar a un documento digital compartido.
- Si solo tienes tiempo para una sección, empieza por el Capítulo 25 (el Framework Canvas) y regresa a la teoría cuando la necesites.

Mapa rápido: qué capítulo leer según tu situación

Vas a construir un producto de datos desde cero	Capítulos 1, 4, 19, 25
Dos personas de tu equipo interpretan el mismo dato distinto	Capítulos 9, 10, 27
Tu modelo o dashboard “falla sin avisar”	Capítulos 6, 12, 14, 23
Vas a diseñar un asistente de IA generativa	Capítulo 20
Estás planeando un lanzamiento con horizonte largo	Capítulos 8, 22, 24
Tu manual de reglas de negocio ya nadie lo entiende	Capítulos 7, 13, 17
Necesitas instalar esta disciplina en tu equipo	Capítulos 27, 28, 29

Índice

PARTE I · EL PROBLEMA QUE NO VEMOS

1. Por qué el framing es la decisión antes de la decisión	01
2. Una máquina que no sabe qué ignorar	02
3. El frame problem: historia, definición y por qué nadie se pone de acuerdo	03
4. Situaciones, fluents y el vocabulario que usas sin saberlo	04
5. Por qué tu empresa tiene el mismo problema que un sistema de 1969	05
6. El costo oculto de los supuestos que no escribimos	06

PARTE II · POR QUÉ ES TAN DIFÍCIL

7. Demasiadas reglas: cuando el manual de operaciones explota	07
8. Concurrencia: el mundo no espera a que termines tu acción	08
9. El Yale Shooting Problem, llevado a negocio	09
10. Cuando dos supuestos válidos chocan: el diamante de Nixon	10
11. Ramificaciones: el efecto dominó que nadie modeló	11
12. La falacia de "si no cambió, sigue igual"	12

PARTE III · EL TOOLKIT

13. Persistencia por default: qué asumir cuando nada más se sabe	13
14. Cierre por explicación: solo cambia lo que tiene una causa	14
15. STRIPS para humanos: listas de qué se agrega y qué se borra	15
16. Motivated Action Theory: prefiere el mundo con menos sorpresas	16
17. Circunscripción práctica: minimizar lo anormal	17
18. Integrar en vez de reinventar: construir sobre lo que ya funciona	18

PARTE IV · DATOS, IA Y NEGOCIO

— — — — —

19. Framear un producto de datos antes de construirlo	19
20. IA generativa: la ventana de contexto es tu situation calculus	20
21. Retail y operaciones: qué persiste, qué cambia, quién lo causa	21
22. Diseñar decisiones a prueba de concurrencia	22
23. Casos: cuando ignorar el framing costó caro	23
24. Casos: cuando framear bien ahorró meses	24

PARTE V · TU PROPIO FRAMING

25. El Framework Canvas: una página para framear cualquier problema	25
26. Checklist HTDQ 2.0: adaptado a proyectos de datos	26
27. Cómo facilitar una sesión de framing con tu equipo	27
28. Errores comunes al framear (y cómo evitarlos)	28
29. De la teoría a la cultura: instalar el hábito de framear	29
30. Cierre: el framing es una decisión, no un descubrimiento	30

APÉNDICE

Framework Canvas — plantilla	A
Fichas de trabajo del Toolkit	A
Glosario	B
Referencias (formato APA)	B
Sobre el autor	B

El problema que no vemos

Fundamentos

“Cada decisión que tomas empieza con una decisión que no notaste: qué dejar afuera del framing.”

CAPÍTULO 1

“El mapa no es el territorio, pero es lo único que llevas contigo cuando decides.”

Por qué el framing es la decisión antes de la decisión

"El mapa no es el territorio, pero es lo único que llevas contigo cuando decides."

Antes de resolver cualquier problema —de negocio, de datos o de vida— tomas una decisión invisible: qué parte del mundo vas a considerar y qué parte vas a dejar fuera. A esa decisión la vamos a llamar, a lo largo de todo este libro, framing, tomando el término directamente de la literatura de inteligencia artificial y ciencia cognitiva en inglés, sin traducirlo como "encuadre" o "marco". Hay una razón deliberada para esta elección de vocabulario: framing no es solo el resultado —el encuadre final que eliges— sino el proceso activo de decidir, y ese matiz de proceso es precisamente lo que este libro quiere que internalices como una práctica, no como un sustantivo pasivo.

El framing casi nunca se hace a propósito. Se hereda del último proyecto parecido, de la costumbre del equipo, de lo que cabe en el dashboard que alguien construyó hace dos años. Y ahí está el problema real: un framing mal hecho no se anuncia como un error de cálculo. No existe una alarma que diga "estás ignorando la variable correcta". Se manifiesta meses después, cuando el modelo predictivo falla justo en el escenario que nadie modeló, cuando la estrategia funciona en el PowerPoint pero no sobrevive al primer trimestre real, o cuando el producto de datos responde con precisión a una pregunta que nadie necesitaba hacer.

Este libro parte de una idea incómoda, tomada prestada de la inteligencia artificial clásica: la parte más difícil de razonar sobre un mundo que cambia no es calcular qué va a pasar. Es decidir, de forma eficiente, qué va a seguir igual y qué requiere atención activa. A ese desafío, formalizado por primera vez en 1969, John McCarthy y Patrick Hayes lo llamaron the frame problem —el problema del framing—

(McCarthy & Hayes, 1969), y durante más de veinticinco años ocupó a algunos de los investigadores más brillantes de la inteligencia artificial simbólica. No lo resolvieron por completo. Pero en el intento dejaron un catálogo de herramientas de razonamiento que, traducido a lenguaje de negocio, es exactamente lo que necesitas para tomar mejores decisiones en un entorno que no deja de moverse.

Conviene ser precisos desde este primer capítulo sobre qué es, técnicamente, el frame problem, porque el resto del libro construye sobre esta definición y regresará a ella una y otra vez. En su forma más literal y original, es el problema de que un sistema lógico que razona sobre acciones necesita, en principio, un axioma explícito por cada combinación de acción y propiedad que esa acción no afecta. Mover un bloque de una mesa a otra no cambia el color del cielo, no cambia el precio del dólar, no cambia quién es el presidente del país —pero un sistema de lógica formal no sabe eso a menos que se lo digas explícitamente, axioma por axioma. Con muchas acciones posibles y muchas propiedades del mundo, el número de axiomas de framing necesarios crece de forma combinatoria: si hay m acciones y n propiedades típicamente estables por acción, se requieren del orden de $m \times n$ axiomas solo para capturar lo obvio (Morgenstern, 1996).

Pero —y este es el segundo nivel de profundidad que quiero que te lleves de este capítulo— el frame problem, entendido correctamente, no es solo "hay demasiados axiomas". McCarthy y Hayes lo identificaron casi de inmediato como un problema representacional: ¿cómo se dice, de manera concisa dentro de un sistema formal, "todo permanece igual excepto lo que sabemos explícitamente que cambia"? Esta pregunta, que suena casi trivial en lenguaje natural, resultó ser una de las más resistentes en la historia de la lógica aplicada a la inteligencia artificial, precisamente porque los sistemas lógicos clásicos —monotónicos, donde agregar información nunca invalida una conclusión previa— no tienen forma nativa de expresar "por default" o "salvo excepción" (Morgenstern, 1996; Shoham, 1988).

La promesa de este libro no es enseñarte lógica de situaciones ni

circunscripción formal en su notación matemática completa. Es tomar más de dos décadas de un debate académico intenso —con héroes, fracasos públicos, la conversión pública de un lógico convencido a antilogicista, y una anécdota sobre un arma cargada que se volvió célebre en toda la disciplina— y convertirlo en un conjunto de herramientas de framing que puedas usar el lunes: para diseñar un producto de datos, para escribir el brief de un modelo de IA generativa, para decidir qué reportar en un comité, o simplemente para tener una conversación más honesta sobre qué está asumiendo tu estrategia sin decirlo en voz alta.

EJEMPLO EXTENDIDO

El dashboard que mentía sin mentir

Un equipo de retail construyó, con mucho cuidado técnico, un dashboard de quiebres de inventario para monitorear disponibilidad por tienda y categoría. Cada número individual en ese dashboard era, en sentido estricto, correcto: la consulta SQL sumaba correctamente las unidades disponibles, el pipeline de datos corría sin errores, las alertas se disparaban exactamente cuando la lógica programada decía que debían dispararse. Y sin embargo, durante casi un trimestre completo, el dashboard le dio al comité de operaciones una imagen sistemáticamente distorsionada de la realidad.

El origen del problema no estaba en ninguna línea de código defectuosa. Estaba en el framing original del proyecto, siete meses antes: cuando el equipo definió qué contaba como "quiebre de inventario", nadie preguntó explícitamente qué pasaba con las tiendas temporalmente cerradas por remodelación. El supuesto implícito —nunca escrito, nunca discutido en ninguna junta de diseño— era que "tienda cerrada" simplemente no ocurría como estado relevante para el negocio. Bajo ese framing, una tienda en remodelación seguía apareciendo en el sistema como una tienda operando normalmente, y cada producto que no se vendía ahí —porque la tienda estaba físicamente cerrada, no porque

hubiera un problema real de abasto— se contaba como un quiebre de inventario.

Cuando la cadena entró a un periodo de remodelaciones simultáneas en catorce tiendas como parte de una renovación de imagen de marca, el dashboard mostró un salto dramático en la tasa de quiebres nacional. El comité, viendo el número, autorizó compras adicionales de emergencia para las categorías "más afectadas", sin saber que gran parte de esa señal era completamente artificial: un efecto del framing original, no de la operación real. El costo no fue solo el capital de trabajo inmovilizado en inventario que no se necesitaba. Fue, también, la credibilidad del equipo de datos, que tuvo que explicar, semanas después, por qué el número en el que se había basado una decisión de compras de varios millones de pesos no reflejaba la realidad que todos asumían que reflejaba.

Este caso ilustra algo central para todo el libro: nadie mintió. Ningún analista manipuló datos, ningún ingeniero cometió un error de programación identificable. El framing mintió por ellos, porque el framing —la decisión de qué flujos capturar y bajo qué definición— nunca se hizo explícito, revisable ni sometido a la pregunta que este libro insiste en hacer una y otra vez: ¿qué estamos asumiendo que se mantiene igual, y qué evidencia tenemos de que ese supuesto sigue siendo cierto?

EJERCICIO 1 — ENCUENTRA TU FRAMING INVISIBLE

1. Elige una decisión reciente en tu equipo (un dashboard, un modelo, una política).
2. Escribe en una frase qué asumiste que se mantenía igual sin revisarlo.
3. Pregúntate: ¿qué evento, si ocurriera mañana, rompería ese supuesto?
4. Anota quién en tu equipo hubiera detectado ese supuesto antes que tú.

Aplícalo a tu proyecto

| “El mapa no es el territorio, pero es lo único que llevas contigo cuando decides.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Todo niño de cinco años sabe que mover una silla no cambia el color del cielo. Un sistema lógico, no.””

Una máquina que no sabe qué ignorar

“Todo niño de cinco años sabe que mover una silla no cambia el color del cielo. Un sistema lógico, no.”

Para entender por qué el frame problem resultó tan resistente durante más de dos décadas de investigación, conviene detenerse en el ejemplo canónico con el que McCarthy y Hayes lo introdujeron: el mundo de bloques. Imagina un sistema que razona usando lógica de primer orden pura —hechos y reglas de inferencia, sin ningún mecanismo especial para el sentido común—. Le dices al sistema que un bloque rojo, al que llamaremos A, está sobre una mesa. Le dices también que puede ejecutar una acción llamada $\text{Puton}(A, B)$, que mueve el bloque A encima de otro bloque B. Ejecutas esa acción. Pregúntale entonces al sistema: ¿sigue siendo rojo el bloque A?

Para cualquier persona, la respuesta es obviamente sí. Pero para un sistema de lógica estricta, la respuesta no está garantizada por la teoría, a menos que exista, explícitamente, un axioma que diga que el color de un bloque no se ve afectado por la acción de moverlo. Sin ese axioma —llamado, técnicamente, un axioma de framing o frame axiom—, el sistema simplemente no puede concluir nada sobre el color de A después de la acción. No es que concluya lo contrario; es que se queda sin poder decir nada, lo cual, para un sistema que necesita actuar sobre esas conclusiones, es igual de inútil que una conclusión equivocada.

El problema, entonces, no es escribir un axioma de framing. Es escribir todos los que se necesitan. El mundo tiene decenas de miles de propiedades que, con toda evidencia, no cambian con cada acción específica: mover el bloque A no cambia el color del cielo, no cambia el precio del dólar, no cambia quién es el gerente de la tienda, no cambia la capital de Francia. Escribir explícitamente, para cada acción posible del sistema, qué propiedades no se ven afectadas, es una tarea combinatoria sin fin práctico. Esta es la versión más literal y original del frame

problem: no un problema filosófico abstracto, sino una explosión de axiomas obvios que, en cualquier dominio mínimamente rico, se vuelve intratable (McCarthy & Hayes, 1969).

Vale la pena notar algo que la mayoría de las explicaciones divulgativas del frame problem omiten: el problema no desaparece si simplemente "le dices al sistema que asuma que todo sigue igual", porque esa instrucción, en lógica clásica monotónica, no tiene forma de expresarse sin, otra vez, enumerar excepciones. Un sistema lógico monotónico solo puede acumular conclusiones; nunca puede decir "concluyo X, pero estoy dispuesto a retractarme si aparece evidencia en contra". Esa capacidad de retractación condicional —fundamental para expresar "por default, algo sigue igual"— requiere un tipo de lógica completamente distinto, la lógica no monotónica, que exploraremos en detalle en la Parte III.

Las organizaciones enfrentan una versión estructuralmente idéntica de este problema, solo que en lugar de axiomas lógicos usan políticas, manuales de operación y reglas de negocio codificadas en sistemas. Cada vez que aparece una excepción —una tienda cerrada, un cliente que compra por dos canales simultáneamente, un proveedor que cambia de razón social— alguien tiene que decidir, en tiempo real y casi siempre bajo presión, si ese caso es cubierto implícitamente por el sistema existente o si rompe un supuesto que nunca se hizo explícito. La tentación natural, como veremos con detalle en el Capítulo 7, es la misma que tuvieron los primeros investigadores de inteligencia artificial: agregar una regla más. Funciona durante un tiempo. Pero, como en la lógica formal, ese camino tiene un límite que no es solo práctico, sino estructural: el número de reglas necesarias para cubrir cada caso crece más rápido que la capacidad de cualquier equipo de mantenerlas coherentes entre sí.

EJEMPLO EXTENDIDO

El motor de precios que olvidó preguntar

Una cadena de electrodomésticos implementó un motor de precios dinámico diseñado para bajar automáticamente el precio de productos con poco movimiento de inventario, con el objetivo de acelerar su rotación antes de que se volvieran obsoletos. El sistema, técnicamente bien construido, monitoreaba la velocidad de venta de cada SKU y aplicaba descuentos progresivos según reglas de elasticidad de precio calibradas cuidadosamente por el equipo de pricing.

El sistema nunca recibió una instrucción explícita sobre cómo tratar productos que estaban en liquidación por mandato legal —por ejemplo, artículos descontinuados por el fabricante que debían venderse antes de una fecha límite regulatoria, ya a su precio piso permitido—. Para cualquier persona del equipo comercial, era obvio que un producto "en liquidación legal con precio piso" era categóricamente distinto de un producto "simplemente poco atractivo para el consumidor". Pero esa distinción, obvia para un humano, nunca se tradujo en una propiedad explícita del sistema. El motor de precios trataba ambos casos como instancias del mismo flujent —velocidad de venta baja— y aplicaba la misma lógica de descuento a los dos.

El resultado: durante varias semanas, el sistema siguió bajando el precio de productos que ya estaban en su piso legal, generando ventas por debajo del costo autorizado y, en algunos casos, exponiendo a la empresa a un riesgo de cumplimiento normativo, porque el precio resultante violaba el acuerdo de liquidación pactado con el proveedor. Cuando finalmente se detectó el patrón —no por una alerta automática, sino porque un gerente de tienda reportó manualmente precios que "no tenían sentido"—, la causa raíz resultó ser exactamente el tipo de omisión que este capítulo describe: nadie había escrito el axioma de framing que distinguía "este tipo de acción de descuento no debe aplicar a este tipo de producto". No era una falla de programación en el sentido convencional. Era una ausencia de framing explícito sobre una excepción que, para cualquier persona con contexto de negocio, era evidente por sí misma.

La lección no es que el equipo debió haber anticipado ese caso específico

desde el diseño original —es imposible anticipar cada excepción posible, y ese es precisamente el punto central del frame problem—. La lección es que el sistema no tenía ningún mecanismo para señalar, de forma proactiva, que estaba operando fuera de los casos para los que había sido explícitamente diseñado. Un sistema bien framed no necesita anticipar todas las excepciones; necesita saber reconocer cuándo está actuando bajo un supuesto que nunca fue verificado para el caso presente.

EJERCICIO 2 — CAZA DE SUPUESTOS OBVIOS

1. Lista tres reglas de negocio que tu equipo nunca ha escrito porque "se entienden solas".
2. Para cada una, imagina el caso límite que la rompería.
3. Decide si vale la pena escribirla explícitamente o dejarla como supuesto vigilado.

Aplicalo a tu proyecto

*“Todo niño de cinco años sabe que mover una silla no cambia el color del cielo.
Un sistema lógico, no.”*

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Veintiséis años de investigación no resolvieron el problema. Pero dejaron el mejor kit de herramientas que existe para pensar sobre el cambio.””

El frame problem: historia, definición y por qué nadie se pone de acuerdo

"Veintiséis años de investigación no resolvieron el problema. Pero dejaron el mejor kit de herramientas que existe para pensar sobre el cambio."

En 1969, John McCarthy y Patrick Hayes publicaron "Some Philosophical Problems from the Standpoint of Artificial Intelligence", un ensayo fundacional que buscaba formalizar el razonamiento de sentido común dentro de la lógica matemática de primer orden (McCarthy & Hayes, 1969). Para razonar sobre acciones y el paso del tiempo, propusieron el cálculo de situaciones (situation calculus): el mundo se representa como una línea discreta y ramificada de "fotografías" —situaciones— conectadas entre sí por acciones. Una situación captura el estado completo del mundo en un instante; una acción transforma una situación en la siguiente. El aparato formal es elegante y, en el papel, poderoso.

El problema apareció casi de inmediato, y McCarthy y Hayes lo reconocieron ellos mismos en el mismo ensayo que introdujo la situation calculus: para que el sistema supiera qué propiedades del mundo se mantenían iguales después de cada acción, había que escribir un axioma de framing por cada combinación relevante de acción y propiedad estable. Con un número razonable de acciones y de propiedades, el número de axiomas necesarios se disparaba de forma combinatoria. A ese problema —el problema de escribir eficientemente la enorme cantidad de cosas que no cambian— la comunidad de inteligencia artificial terminó llamándolo, simplemente, the frame problem.

Aquí conviene hacer una pausa importante, porque es un punto que la propia literatura académica señala como fuente de buena parte de la confusión posterior: casi nadie define el frame problem exactamente de la misma manera (Morgenstern, 1996). En su forma más restringida, es el problema técnico de reducir el número de axiomas de framing dentro

de la situation calculus. En una forma ligeramente más general, se le identifica con el problema de la persistencia (Shoham, 1988): predecir qué propiedades se mantienen iguales al paso del tiempo, dentro de cualquier formalismo temporal razonable, no solo la situation calculus. En una forma más amplia todavía, se le agrupa junto con su problema dual —predecir qué cambia, no solo qué persiste— bajo el nombre de proyección temporal hacia adelante (forward temporal projection) (Morgenstern & Stein, 1988). Y en su forma más general y, según algunos filósofos, ya desbordada, se le ha llegado a identificar con el problema general de la inducción en su totalidad (Fetzer, 1991) —una generalización que Morgenstern (1996) considera un error categorial, porque bajo ese criterio, prácticamente cualquier forma de razonamiento sobre el mundo, incluyendo el razonamiento probabilístico ordinario, podría llamarse igualmente "el frame problem".

Durante los años ochenta y noventa, decenas de investigadores propusieron soluciones formales al frame problem, casi todas dentro de la tradición de la lógica no monotónica. El patrón que se repitió, con una regularidad casi cómica si no fuera tan frustrante para quienes la vivieron, fue el siguiente: se proponía una solución; alguien encontraba un contraejemplo, generalmente una variación mínima sobre un problema de juguete ya conocido, que la solución no lograba resolver correctamente; se proponía una nueva solución, o una corrección a la anterior; y el ciclo volvía a empezar. El episodio más célebre de este patrón —y el que le dio al debate una anécdota memorable que sobrevive hasta hoy en la literatura— es el llamado Yale Shooting Problem, que dedicaremos el Capítulo 9 a desarrollar con el detalle que merece.

¿Por qué te debería importar esta historia si nunca vas a escribir un axioma de lógica formal? Porque el patrón estructural —proponer una regla general, encontrar el caso que la rompe, parchear, repetir— es exactamente el mismo patrón que viven, sin llamarlo así, los equipos de datos, producto y operaciones en cualquier organización moderna. Entender la historia del frame problem no te da una fórmula mágica que la propia academia tampoco logró producir en su forma más general. Pero sí te da algo de valor práctico considerable: un vocabulario preciso

para nombrar por qué tus reglas de negocio siguen fallando de formas similares entre sí, y un catálogo razonablemente completo de estrategias parciales que sí funcionaron dentro de ciertos límites bien entendidos, y que puedes adaptar con criterio a tu propio dominio.

EJEMPLO EXTENDIDO

De la lógica de bloques al catálogo de Netflix

Cuando un equipo de producto diseña qué pasa con la cuenta de un usuario que cancela su suscripción a un servicio de streaming, está resolviendo, sin llamarlo así, una instancia genuina del frame problem. La decisión no es trivial: ¿qué persiste después de la acción de cancelación, y qué deja de existir?

Considera las opciones de diseño reales que un equipo así enfrenta. El historial de visualización —qué series vio el usuario, en qué episodio se quedó— podría tratarse como un fluent que persiste indefinidamente, o como uno que se elimina al cancelar. Las recomendaciones personalizadas, construidas sobre ese historial, dependen de la decisión anterior. El acceso al catálogo, evidentemente, debe cambiar de "disponible" a "no disponible" con la cancelación —esa es la acción principal, el equivalente al Puton del mundo de bloques—. Pero decisiones menos obvias, como si el usuario sigue recibiendo notificaciones de nuevos estrenos en géneros que solía ver, o si su método de pago permanece guardado para una eventual reactivación, requieren axiomas de framing igual de explícitos, aunque nadie en el equipo de producto los llame así.

Netflix, según se ha discutido en foros de diseño de producto y documentación pública de sus prácticas de ingeniería, tomó la decisión de que el historial de visualización persiste durante un periodo extendido después de la cancelación, incluso si el acceso al contenido no. Esta no es una decisión técnica trivial: requiere almacenamiento de datos de un usuario que ya no genera ingreso directo, y expone a la empresa a preguntas de privacidad y retención de datos. Pero la decisión de diseño

explícita —persistencia del historial, no persistencia del acceso— evitó un problema mucho más costoso: usuarios que reactivaban su cuenta meses después y descubrían que toda su experiencia personalizada, construida durante años, había desaparecido, generando fricción y abandono en el momento crítico de reactivación.

El paralelo con el mundo de bloques es exacto, aunque el vocabulario sea distinto. "Cancelar suscripción" es la acción; "historial de visualización", "acceso al catálogo", "método de pago guardado" son flujos; y la pregunta de diseño de producto —¿qué persiste, qué cambia, y quién lo decide explícitamente en lugar de dejarlo a la implementación por default de cada sistema involucrado?— es, palabra por palabra, la pregunta que McCarthy y Hayes formalizaron en 1969 para un mundo de bloques de juguete.

Aplicalo a tu proyecto

“Veintiséis años de investigación no resolvieron el problema. Pero dejaron el mejor kit de herramientas que existe para pensar sobre el cambio.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Toda estrategia es, en el fondo, una fotografía de un momento que ya pasó.””

Situaciones, fluents y el vocabulario que usas sin saberlo

"Toda estrategia es, en el fondo, una fotografía de un momento que ya pasó."

La situation calculus descompone el mundo en tres conceptos que conviene definir con precisión, porque se convertirán en el vocabulario de trabajo de todo el libro. Una situación (situation) es una fotografía completa del mundo en un instante determinado: el equivalente a decir "así estaban las cosas exactamente a la 1:00 p.m. del 17 de julio". Un fluent es cualquier propiedad del mundo cuyo valor puede cambiar con el tiempo: si una tienda está abierta, cuál es el precio de un producto, quién ocupa un cargo determinado. Los fluents booleanos —verdadero o falso— se llaman típicamente estados (states); los fluents que toman un valor entre varios posibles —como "presidente de la República", que toma el nombre de una persona— se modelan como términos que varían en el tiempo (McCarthy & Hayes, 1969; Morgenstern, 1996). Y una acción es, precisamente, lo que transforma una situación en la siguiente: la función `Result(acción, situación)` devuelve la nueva situación resultante de ejecutar esa acción sobre la situación dada.

Esta descomposición, aunque nació en un laboratorio de lógica computacional en los años sesenta, es exactamente el vocabulario que usa —casi siempre sin saberlo— cualquier equipo que arma un dashboard, entrena un modelo predictivo o redacta un plan estratégico. Cuando tu equipo de datos define un "corte" mensual de información, está declarando, sin el nombre técnico, una situación. Cuando define qué campos pueden cambiar de un corte a otro —inventario, precio, estatus del cliente—, está declarando fluents. Y cuando modela qué eventos disparan esos cambios —una venta, una devolución, un cierre de tienda—, está declarando acciones en el sentido preciso de la situation calculus.

El problema práctico es que casi ningún equipo hace esta descomposición de manera consciente. La hace de forma implícita, campo por campo, decisión por decisión, sin preguntarse nunca de forma explícita: ¿qué asumimos que permanece igual entre un corte y el siguiente, y sobre qué evidencia se sostiene ese supuesto? Esta falta de explicitud no es un defecto de disciplina personal; es, casi siempre, un problema estructural del proceso de diseño, que rara vez incluye un paso dedicado exclusivamente a esta pregunta.

Aquí está la primera herramienta operativa del libro, y volverá una y otra vez en capítulos posteriores: separa, de forma explícita, para cualquier problema que estés modelando, tres listas distintas de fluents. La primera, los fluents volátiles: propiedades que cambian con casi cada acción relevante, y que por lo tanto deben capturarse con precisión y frecuencia en cualquier sistema que las use. La segunda, los fluents condicionales: propiedades que cambian, pero solo bajo condiciones específicas y relativamente infrecuentes —un cambio de política de precios, por ejemplo—. La tercera, los fluents estables asumidos: propiedades que el proyecto trata como constantes dentro de su horizonte de decisión, sin necesariamente verificarlas de forma activa en cada ciclo.

La mayoría de los errores de framing documentados en este libro —y, según mi experiencia trabajando en arquitecturas de datos y analítica en organizaciones de gran escala, la mayoría de los que vas a encontrar en tu propia carrera— ocurren en la frontera entre la segunda y la tercera lista: algo que un equipo trató como estable resultó ser, en realidad, condicional, y nadie lo puso por escrito antes de que el supuesto se rompiera en producción, frente a un cliente, o en medio de una junta directiva.

EJEMPLO EXTENDIDO

Las tres listas de un modelo de churn

Un equipo de datos de una empresa de telecomunicaciones construyó un

modelo de predicción de cancelación de clientes (churn) usando dos años de historia transaccional. Para diseñarlo correctamente desde el framing, el equipo debía clasificar cada variable candidata en una de las tres listas del capítulo: volátil, condicional o estable.

En la lista de flujos volátiles colocaron, correctamente, la actividad reciente del cliente: llamadas de los últimos 30 días, uso de datos móviles, tickets de soporte abiertos. Estas variables se recalculaban en cada corte y alimentaban directamente el modelo. En la lista de flujos condicionales colocaron el plan de precios contratado por cada cliente, correctamente identificado como algo que cambia, pero solo bajo un evento específico y relativamente infrecuente: una renegociación comercial formal. Hasta aquí, el framing del proyecto era sólido.

El problema apareció en la tercera lista, la de flujos estables asumidos. El equipo trató como estable, sin discutirlo explícitamente con el área de negocio, la definición operativa de "cliente activo" —la variable objetivo misma que el modelo intentaba predecir que dejaría de serlo—. Esa definición llevaba tres años sin cambios y, en la memoria colectiva del equipo, se había convertido en un hecho del mundo, no en una decisión de negocio revisable. Cuando el área comercial, sin coordinarse con el equipo de datos, redefinió "cliente activo" como parte de una nueva estrategia de retención —ampliando la ventana de inactividad tolerada de 30 a 60 días antes de considerar a alguien inactivo—, el modelo de churn siguió operando con la definición anterior durante seis semanas completas.

El resultado fue un desalineamiento silencioso: el modelo predecía churn usando una definición de "activo" que ya no coincidía con la que usaba el resto de la organización para reportar resultados a la dirección. Las cifras de churn predicho y churn reportado divergieron de forma creciente, generando confusión en el comité de retención durante semanas, hasta que alguien —por casualidad, revisando ambas definiciones lado a lado en una reunión de alineación— conectó las dos cosas. El costo no fue solo el tiempo de diagnóstico; fue también la credibilidad del modelo frente al comité, que durante ese periodo dejó

de confiar en sus predicciones sin entender bien por qué. La causa raíz, una vez más, no fue un error técnico. Fue tratar como estable un fluent que, en realidad, era condicional a decisiones de negocio que el equipo de datos no controlaba ni monitoreaba activamente.

EJERCICIO 3 — LAS TRES LISTAS

1. Toma el modelo de datos o dashboard más importante de tu equipo.
2. Clasifica sus campos clave en volátil, condicional y estable.
3. Marca en rojo los que llevan más de un año sin ser revisados.

Aplícalo a tu proyecto

“Toda estrategia es, en el fondo, una fotografía de un momento que ya pasó.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““La escala cambia. La estructura del problema,
no.””

Por qué tu empresa tiene el mismo problema que un sistema de 1969

“La escala cambia. La estructura del problema, no.”

Es tentador pensar que el frame problem es un curioso capítulo cerrado de la historia de la inteligencia artificial simbólica, resuelto hace tiempo por arquitecturas más modernas, sin relevancia real para el mundo de los negocios de hoy. Es un error, y vale la pena mostrarlo con algo de detalle técnico. Los grandes modelos de lenguaje contemporáneos —los mismos que probablemente usas todos los días para escribir, analizar o programar— no resuelven el frame problem; heredan una versión moderna, estadística y distribuida del mismo problema estructural. No saben, de forma confiable y verificable, qué parte del contexto que se les proporcionó sigue siendo relevante después de veinte turnos de conversación, y qué parte ya quedó obsoleta porque el usuario la corrigió tres mensajes atrás. La "ventana de contexto" de un modelo de lenguaje es, funcionalmente, un intento —imperfecto y sin garantías formales— de resolver el mismo problema de persistencia que ocupó a McCarthy, Hayes y sus sucesores durante décadas: qué información sigue siendo válida y cuál debe descartarse o actualizarse.

Pero incluso si tu organización nunca tocara un modelo de inteligencia artificial, el problema seguiría ahí, porque no es, en el fondo, un problema de tecnología específica. Es un problema de razonar sobre el tiempo con recursos cognitivos y computacionales limitados. Toda organización es, en esencia, un sistema que necesita predecir, con recursos finitos de atención y análisis, qué se mantiene igual y qué cambia, para poder actuar con algo de anticipación en lugar de reaccionar constantemente a sorpresas. Un plan estratégico a tres años es, en un sentido literal y no metafórico, una apuesta sobre qué flujos del mercado se mantendrán estables y cuáles cambiarán —y, más sutilmente, sobre cuáles de esos cambios la propia empresa va a causar

de forma controlada versus cuáles va a sufrir como acciones concurrentes fuera de su control, el tema que desarrollaremos en el Capítulo 8.

La diferencia observable entre una empresa que navega bien la incertidumbre estructural de su mercado y una que se rompe con cada sorpresa razonable no es, contra la intuición común, que la primera tenga mejores pronósticos numéricos. Los pronósticos, en la mayoría de los entornos de negocio genuinamente complejos, tienen un techo de precisión que ningún equipo, por sofisticado que sea su modelado estadístico, puede superar de forma sostenida. La diferencia real está en que la primera empresa tiene mejor framing por default: sabe, con mayor precisión y mayor consenso interno, qué asumir que sigue igual mientras no haya evidencia específica de lo contrario, y sabe reconocer más rápido, con menos fricción organizacional, cuándo esa evidencia efectivamente apareció.

Esta distinción —entre mejorar la predicción y mejorar el framing— es, en mi experiencia, sistemáticamente subestimada en las conversaciones sobre estrategia de datos e inteligencia artificial en las organizaciones. Se invierte una cantidad enorme de presupuesto y talento en mejorar modelos predictivos, y una fracción mínima en mejorar la disciplina de framing que determina si esos modelos están, para empezar, respondiendo a la pregunta correcta bajo los supuestos correctos. El resto de este libro toma las estrategias que la inteligencia artificial simbólica desarrolló para atacar exactamente este problema —minimización de anormalidad, persistencia por default, cierre por explicación, motivación de acciones— y las traduce en prácticas de framing instalables en tu equipo, sin necesitar una sola línea de notación lógica formal.

EJEMPLO EXTENDIDO

Dos empresas, la misma disrupción, dos resultados

Durante una disrupción significativa en una cadena de suministro

regional —el tipo de evento que, según discutiremos en el Capítulo 8, constituye una acción concurrente fuera del control de cualquier empresa individual—, dos cadenas de retail de tamaño comparable, operando en la misma categoría y geografía, enfrentaron el mismo choque externo: un incremento súbito y sostenido en el costo de un insumo crítico para varias de sus líneas de producto más vendidas.

La primera cadena tenía, como parte de su proceso de planeación trimestral, un documento de framing explícito —aunque no lo llamaran así— que identificaba el costo de ese insumo específico como un flujante condicional de alto riesgo, con un punto de revisión programado a mitad de cada trimestre, específicamente diseñado para verificar si el supuesto de estabilidad de costos seguía siendo válido. Cuando el choque ocurrió, el punto de revisión ya estaba en el calendario, con un responsable nombrado y una ruta de ajuste predefinida —renegociación de contratos alternativos, ajuste de precio al consumidor en un rango preaprobado por el comité comercial—. La cadena activó la ruta de ajuste en menos de una semana desde que la señal apareció en sus reportes internos.

La segunda cadena, de tamaño y sofisticación analítica comparables, no tenía ningún mecanismo equivalente. Su plan trimestral asumía, de forma completamente implícita —nunca discutida, nunca escrita como supuesto revisable—, que los costos de insumos se mantendrían dentro de un rango histórico normal durante todo el trimestre. Cuando el mismo choque de costos ocurrió, la señal apareció primero en el estado de resultados mensual, casi seis semanas después del evento original, procesada a través del ciclo normal de reportería financiera, no a través de ningún mecanismo de detección temprana diseñado específicamente para ese riesgo. Para cuando el comité directivo discutió el problema, el margen erosionado ya representaba varias semanas de operación con rentabilidad significativamente por debajo del objetivo, y la respuesta —ya reactiva, ya bajo presión— tomó considerablemente más tiempo en implementarse que la de la primera cadena.

La diferencia entre ambas no fue el acceso a datos —las dos tenían sistemas de reportería financiera y operativa de calidad comparable—.

Fue que la primera había hecho explícito, con antelación, exactamente qué supuesto de persistencia estaba en juego, y había diseñado un mecanismo específico para vigilarlo. La segunda dependía de que la sorpresa fuera, eventualmente, lo suficientemente grande como para abrirse paso por los canales genéricos de reportería. El frame problem, en este caso, no se resolvió con mejor tecnología. Se resolvió con mejor disciplina de framing aplicada a un riesgo específico, identificado con antelación.

Aplícalo a tu proyecto

| “La escala cambia. La estructura del problema, no.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“No pagas por los supuestos que haces. Pagas por los que no sabías que estabas haciendo.”

El costo oculto de los supuestos que no escribimos

"No pagas por los supuestos que haces. Pagas por los que no sabías que estabas haciendo."

Cada supuesto no escrito tiene un costo económico real, y ese costo no aparece en ningún estado de resultados hasta el momento en que el supuesto se rompe. Entonces aparece de golpe, casi siempre disfrazado de crisis puntual en lugar de reconocerse como el síntoma estructural que realmente es: el proyecto que se retrasa varios meses porque "nadie consideró" que el proceso de aprobación regulatoria cambiaría a mitad de camino, el modelo que hay que reentrenar de emergencia porque "nadie consideró" que la definición de una métrica clave cambiaría a mitad de año fiscal, el producto de inteligencia artificial generativa que responde con información de hace seis meses porque "nadie consideró" diseñar un mecanismo de actualización del contexto que se le inyecta al modelo.

Hay una asimetría económica importante que conviene internalizar, porque es la que justifica, en términos puramente de retorno sobre inversión de tiempo, todo el esfuerzo metodológico que propone este libro: escribir un supuesto de framing explícito cuesta, típicamente, minutos de discusión en equipo. Descubrir que tenías un supuesto implícito equivocado —en producción, frente a un cliente enojado, o en medio de un comité directivo que exige explicaciones— cuesta, de forma consistente en los casos que he observado, semanas de trabajo de diagnóstico y remediación, además de un costo de credibilidad organizacional que rara vez se contabiliza pero que afecta directamente cuánta autonomía y presupuesto recibe un equipo en el futuro.

Sin embargo —y este matiz es tan importante como la asimetría misma— la respuesta correcta no es intentar escribir todos los supuestos posibles de cualquier proyecto, porque eso nos devuelve exactamente al

problema combinatorio original descrito en el Capítulo 2: la explosión de axiomas de framing. El arte genuino, y la disciplina concreta que este libro busca instalar capítulo a capítulo, está en decidir con criterio qué supuestos vale la pena hacer explícitos —porque su ruptura sería costosa, silenciosa o ambas cosas— y cuáles se pueden dejar razonablemente implícitos, porque su ruptura sería obvia de inmediato, barata de corregir, o simplemente improbable dentro del horizonte temporal relevante de la decisión.

Esa decisión de priorización —qué hacer explícito y qué dejar implícito, con qué nivel de vigilancia activa para cada categoría— es, resumida en una frase, el frame problem aplicado a la práctica organizacional cotidiana. El resto de este libro es, en un sentido preciso, un manual extendido para tomar esa decisión de priorización de forma más sistemática y menos dependiente de la suerte o de la memoria individual de una sola persona en el equipo.

Vale la pena además distinguir dos tipos de costo que se derivan de un supuesto roto, porque requieren respuestas organizacionales distintas. El primero es el costo de remediación técnica: el tiempo y recurso necesarios para corregir el sistema, modelo o proceso una vez detectado el problema. El segundo, con frecuencia más caro y más difícil de revertir, es el costo de confianza: una vez que un comité directivo o un cliente descubre que una decisión importante se tomó sobre un supuesto que resultó falso y que nadie había verificado, la disposición de esa audiencia a confiar en las próximas recomendaciones del mismo equipo se deteriora, incluso si el equipo corrige el problema técnico con rapidez y competencia.

EJEMPLO EXTENDIDO

El supuesto de cuarenta millones de pesos

Una cadena de tiendas de conveniencia con presencia nacional construyó, con apoyo de un equipo externo de consultoría en optimización operativa, un modelo de dotación óptima de personal por

tienda, diseñado para minimizar el costo laboral total manteniendo un nivel de servicio objetivo medido en tiempo de espera en caja. El modelo, técnicamente sofisticado, incorporaba estacionalidad, patrones de tráfico por hora y por día de la semana, y elasticidad de conversión de ventas respecto al nivel de servicio.

El proyecto de construcción del modelo, sin embargo, nunca incluyó una sesión de framing explícita sobre qué variables del entorno laboral se asumían estables durante el horizonte de planeación del modelo, fijado en doce meses. Una de esas variables, tratada implícitamente como estable porque llevaba varios años sin cambios significativos, era la tasa de rotación de personal en tienda: cuántos empleados dejaban su puesto en un periodo dado, una variable que afecta directamente el costo real de mantener el nivel de dotación planeado, porque cada salida implica un periodo de vacante, un proceso de reclutamiento y un periodo de curva de aprendizaje del nuevo empleado durante el cual la productividad es menor a la planeada.

Ocho meses después del despliegue del modelo, un cambio regulatorio laboral significativo —relacionado con condiciones de contratación en el sector retail— disparó la tasa de rotación de personal en 18 puntos porcentuales respecto al nivel histórico sobre el cual el modelo había sido calibrado. El modelo, que no tenía ningún mecanismo de detección de este tipo de cambio estructural en sus supuestos base, continuó optimizando la dotación de personal usando parámetros de rotación completamente obsoletos, generando de forma simultánea dos problemas: sobredotación en algunas tiendas, donde el modelo compensaba de forma imprecisa una rotación que no había sido actualizada en sus parámetros, y quiebres de servicio en otras, donde la combinación de vacantes no cubiertas y curvas de aprendizaje subestimadas generó tiempos de espera muy por encima del objetivo.

La combinación de sobredotación y quiebres de servicio, estimada retrospectivamente por el equipo financiero de la cadena, generó un costo total superior a los cuarenta millones de pesos a lo largo de los ocho meses en que el desalineamiento pasó sin detectarse. La causa raíz,

una vez diagnosticada, no fue un error en el modelo estadístico —el modelo, dado los parámetros con los que había sido calibrado, funcionaba exactamente como se esperaba—. Fue la ausencia de un mecanismo explícito de framing que hubiera identificado la tasa de rotación como un fluent condicional de alto riesgo, sujeto a cambios regulatorios externos, y hubiera establecido un punto de revisión periódico específicamente diseñado para verificar si el supuesto seguía siendo válido, en lugar de asumirlo como una constante estructural del negocio.

EJERCICIO 4 — AUDITORÍA DE SUPUESTOS SILENCIOSOS

1. Reúne a tu equipo y pide que cada persona escriba, en cinco minutos, tres supuestos que "todos dan por hecho" sobre el proyecto actual.
2. Compara las listas: los supuestos que no coinciden entre personas son los de mayor riesgo.
3. Prioriza por costo de ruptura, no por probabilidad de ruptura.

Aplicalo a tu proyecto

“No pagas por los supuestos que haces. Pagas por los que no sabías que estabas haciendo.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

CIERRE DE PARTE I

Antes de continuar

¿Qué supuesto invisible identificaste en tu propio equipo al leer esta parte?

TU RESPUESTA

UNA ACCIÓN CONCRETA PARA ESTA SEMANA

Por qué es tan difícil

Diagnóstico

“No es que falte inteligencia. Es que el mundo cambia más rápido de lo que podemos escribir reglas.”

“Si necesitas mil reglas para describir un mundo simple, el problema no es el mundo. Es cómo lo estás describiendo.”

Demasiadas reglas: cuando el manual de operaciones explota

“Si necesitas mil reglas para describir un mundo simple, el problema no es el mundo. Es cómo lo estás describiendo.”

La primera respuesta natural al frame problem, tanto en lógica formal como en el diseño de sistemas de negocio, es agregar más reglas. Si el sistema falla en un caso, se agrega una excepción. Si falla en otro, se agrega otra. A este enfoque, la literatura académica lo llama el enfoque monotónico, precisamente porque cada regla nueva se suma al conjunto existente sin nunca retirar ni reconsiderar nada previamente establecido —una propiedad formal de la lógica clásica, donde el conjunto de conclusiones derivables solo puede crecer conforme se agregan premisas, nunca reducirse (Morgenstern, 1996)—.

Este enfoque tiene un límite matemático preciso, no meramente práctico. Como se estableció en el Capítulo 2: si hay m acciones posibles dentro de un sistema y aproximadamente n propiedades que típicamente no se ven afectadas por cada acción, el número de axiomas de framing necesarios para cubrir explícitamente todas las combinaciones es del orden de m multiplicado por n (McCarthy & Hayes, 1969; Morgenstern, 1996). En un dominio con cien acciones posibles y cien propiedades relevantes del mundo —una escala modesta para cualquier sistema de negocio real, que fácilmente puede tener miles de cada uno—, esto implica potencialmente diez mil axiomas o reglas explícitas solo para expresar, en esencia, "casi todo permanece igual con cada acción".

En una empresa mediana, la manifestación de este mismo límite matemático es el manual de operaciones que empezó con veinte páginas razonables y hoy tiene cuatrocientas, con excepciones anidadas dentro de otras excepciones, referenciando casos particulares que ocurrieron una sola vez, años atrás, y que nadie en el equipo actual recuerda con precisión el contexto original. O el motor de reglas de negocio que

ningún ingeniero se atreve a tocar, porque ya nadie entiende con certeza por qué existe cada regla individual, solo que quitarla rompe algo, en algún caso de borde que ocurrió alguna vez y que el sistema de pruebas automatizadas —si acaso existe uno robusto— apenas logra capturar.

El síntoma diagnóstico de que un equipo está atrapado en esta trampa no es, contra lo que podría pensarse, la cantidad absoluta de reglas en el sistema —algunos dominios genuinamente complejos requieren, de forma legítima, un número considerable de reglas explícitas—. El síntoma real es la relación entre reglas nuevas agregadas y casos efectivamente resueltos de forma duradera. Cuando cada caso nuevo que aparece requiere, sistemáticamente, una regla nueva específica para ese caso, en lugar de ser absorbido por un principio general ya existente en el sistema, ese sistema de reglas está creciendo de forma puramente monotónica, y el *frame problem* le está cobrando intereses compuestos, en el sentido literal de que cada regla nueva incrementa la probabilidad de conflicto o redundancia con alguna de las reglas ya existentes.

La alternativa correcta no es "tener menos reglas" de forma ingenua y sin criterio —una simplificación excesiva puede ser tan peligrosa, en la dirección opuesta, como la proliferación descontrolada—. La alternativa es cambiar el tipo de regla que se escribe: en lugar de reglas que enumeran excepciones de forma exhaustiva, reglas que declaran principios generales por default, con mecanismos explícitos y bien definidos para anular esos defaults cuando corresponda. Ese cambio de paradigma —de la enumeración exhaustiva a la persistencia por default con anulación explícita— es exactamente lo que la inteligencia artificial descubrió al abandonar progresivamente la lógica monotónica pura en favor de la lógica no monotónica, y es el tema central de la Parte III de este libro.

EJEMPLO EXTENDIDO

El motor de reglas de mil doscientas líneas

Un equipo de riesgo crediticio de una institución financiera regional

heredó, con el tiempo, un motor de reglas de aprobación de crédito que había crecido de forma orgánica durante seis años, acumulando reglas condicionales agregadas por sucesivos equipos, cada uno respondiendo a un caso de excepción específico que había surgido en su momento: un tipo particular de fraude detectado, un segmento de clientes con comportamiento atípico, un requisito regulatorio nuevo que aplicaba solo a cierto tipo de producto crediticio.

Cuando un nuevo líder de riesgo asumió la responsabilidad del área y decidió auditar el sistema completo antes de proponer mejoras, el motor de reglas contenía más de mil doscientas condiciones lógicas interconectadas. La auditoría, realizada durante casi dos meses con apoyo de un equipo dedicado, arrojó hallazgos preocupantes en varias dimensiones. Aproximadamente el cuarenta por ciento de las reglas no se había activado ni una sola vez en los últimos veinticuatro meses de operación, lo cual sugería que respondían a condiciones de mercado o patrones de fraude que ya no eran relevantes, pero que nadie se había animado a retirar por temor a una excepción no documentada que dependiera de ellas.

Más preocupante todavía, el equipo de auditoría identificó que aproximadamente un quince por ciento de las reglas contenía contradicciones lógicas directas con al menos otra regla del sistema — condiciones que, bajo ciertas combinaciones específicas de variables de entrada, producían resultados de aprobación y rechazo simultáneos, resueltos únicamente por el orden arbitrario en que el motor evaluaba las reglas, un detalle de implementación que ningún analista de negocio había diseñado deliberadamente ni podía explicar cuando se le preguntaba—. El sistema "funcionaba", en el sentido de que producía decisiones consistentes día a día, únicamente porque esas contradicciones, por coincidencia estadística acumulada a lo largo de los años, resultaban con mayor frecuencia en el mismo resultado final que se habría obtenido sin la contradicción, no porque el diseño lógico fuera coherente.

El proceso de remediación, una vez identificado el problema, no

consistió en agregar reglas nuevas para corregir las contradicciones encontradas —el reflejo natural, y el mismo reflejo que había producido el problema original—. Consistió en el proceso inverso: identificar un pequeño conjunto de principios generales de aprobación por default, respaldados por el análisis de riesgo histórico agregado, y reclasificar cada una de las mil doscientas reglas originales en una de dos categorías: excepciones genuinamente necesarias, respaldadas por evidencia de riesgo específica y documentada, o resabios históricos de un caso ya obsoleto, candidatos a retiro. Al final del proceso, el sistema quedó reducido a menos de ciento cincuenta reglas activas, con una cobertura de riesgo equivalente —verificada mediante backtesting contra la historia de aprobaciones— a la del sistema original de mil doscientas.

EJERCICIO 5 — PRUEBA DE CRECIMIENTO MONOTÓNICO

1. Revisa el histórico de cambios de tu sistema de reglas o política más compleja.
2. Cuenta cuántos cambios de los últimos 12 meses fueron "agregar excepción" versus "simplificar con un principio general".
3. Si la proporción es mayor a 4 a 1 a favor de excepciones, tienes una señal de alarma.

Aplicalo a tu proyecto

“Si necesitas mil reglas para describir un mundo simple, el problema no es el mundo. Es cómo lo estás describiendo.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Mientras decides, el mundo sigue decidiendo por su cuenta.””

Concurrencia: el mundo no espera a que termines tu acción

“Mientras decides, el mundo sigue decidiendo por su cuenta.”

Casi todas las primeras soluciones formales al *frame problem* compartían un supuesto conveniente para la matemática, pero falso para casi cualquier dominio real: que solo ocurre una acción a la vez, y que el sistema que razona conoce todas las acciones relevantes que ocurren en cada momento. La *situation calculus* original, tal como la formularon McCarthy y Hayes, está construida de forma nativa sobre este supuesto de secuencialidad estricta y conocimiento completo (Gelfond, Lifschitz, & Rabinov, 1991, discuten con detalle las limitaciones que esto impone). Bajo ese supuesto, es relativamente tratable razonar sobre qué cambia y qué no con cada acción individual, precisamente porque no hay que considerar interacciones entre acciones simultáneas.

El problema, evidentemente, es que el mundo real —y cualquier negocio real que opere en él— está lleno de acciones concurrentes, y una fracción significativa de ellas resulta además desconocida para quien está tomando una decisión específica en un momento dado. Piensa en cualquier decisión de negocio de cierta relevancia que tomes hoy: mientras tu organización la implementa a lo largo de semanas o meses, docenas de otras acciones relevantes están ocurriendo en paralelo, muchas de ellas completamente invisibles para ti en el momento de decidir. Un competidor lanza una promoción agresiva. Un proveedor renegocia sus términos con otro cliente, afectando indirectamente su disponibilidad para ti. Un regulador publica una norma nueva que altera el marco de cumplimiento aplicable a tu industria. Tu estrategia, diseñada bajo el supuesto implícito —casi nunca articulado explícitamente— de que “nada más importante va a pasar mientras tanto”, tiene que sobrevivir, sin embargo, a un mundo que en ningún momento se detuvo a esperar tu ejecución.

Los sistemas de inteligencia artificial que asumían acciones únicas y completamente conocidas —como el algoritmo de planeación STRIPS, que estudiaremos con detalle en el Capítulo 15— eran extraordinariamente eficientes computacionalmente dentro de ese supuesto restrictivo, y completamente frágiles fuera de él (Fikes & Nilsson, 1971; Pednault, 1991). La lección para el diseño de decisiones de negocio es directa y merece decirse sin rodeos: cualquier framework de decisión que solo funcione correctamente bajo el supuesto de "nada más relevante está pasando simultáneamente" es, en el mejor de los casos, un framework de laboratorio controlado, no un framework de campo listo para operar en condiciones reales. Y sin embargo, es exactamente el supuesto implícito que hacen la mayoría de los planes estratégicos convencionales, escritos con una estructura lineal que presupone, aunque nadie lo diga en voz alta, que el mercado se va a mantener razonablemente quieto mientras la organización ejecuta su plan.

La pregunta operativamente útil, entonces, no es "¿cómo elimino la concurrencia de mi entorno de decisión?" —no puedes, y ningún framework, por sofisticado que sea, te lo va a permitir—, sino "¿qué tan frágil es esta decisión específica ante acciones concurrentes que no controlo ni conozco con anticipación?". Diseñar de forma deliberada para la concurrencia significa construir, desde el diseño inicial del plan, puntos de revisión explícitos y con dueño asignado, en lugar de simplemente asumir, por default y sin evidencia, que el plan original seguirá siendo válido solo porque nadie ha avisado formalmente lo contrario. Desarrollaremos esta técnica de diseño con detalle operativo en el Capítulo 22.

EJEMPLO EXTENDIDO

El lanzamiento que ignoró al resto del mundo

Una marca regional de productos de consumo masivo planeó, con varios meses de anticipación y un proceso de aprobación interno riguroso, el

lanzamiento de una nueva línea de producto premium, con un presupuesto de marketing y distribución significativo comprometido para un trimestre completo de ejecución. El plan financiero del lanzamiento, construido con detalle considerable, incluía proyecciones de margen basadas en una estructura de costos que asumía, de forma completamente implícita, estabilidad en el tipo de cambio durante el periodo de ejecución —una parte relevante de los insumos del nuevo producto se importaba, facturados en moneda extranjera—.

El plan de lanzamiento, revisado en múltiples instancias de aprobación por distintos comités internos, nunca incluyó ningún punto de revisión intermedio específicamente dedicado a verificar la validez de ese supuesto cambiario durante la ejecución. La estructura del plan era, en ese sentido, puramente lineal: aprobación inicial, ejecución continua durante el trimestre, evaluación de resultados al cierre. No existía ningún mecanismo diseñado para capturar información relevante que pudiera invalidar los supuestos originales antes de llegar a ese cierre trimestral final.

A mitad del trimestre de ejecución, un movimiento cambiario significativo del orden del doce por ciento —una acción completamente concurrente y externa al control de la marca, disparada por factores macroeconómicos que no tenían relación directa con la categoría de producto ni con la industria específica de la empresa— alteró de forma sustancial la estructura de costos del producto recién lanzado. El equipo de finanzas detectó el efecto en el reporte de cierre mensual regular, aproximadamente seis semanas después de que el movimiento cambiario hubiera comenzado a afectar los costos reales de reposición de inventario.

Para cuando el comité directivo discutió formalmente el problema, el margen proyectado original para el lanzamiento se había erosionado en una proporción cercana al total, y las opciones de respuesta disponibles —ajuste de precio al consumidor, renegociación urgente con proveedores, revisión del plan de distribución para el resto del trimestre— resultaban considerablemente más costosas y disruptivas de

implementar bajo presión de tiempo que si se hubieran evaluado con antelación, en un punto de revisión diseñado específicamente para este tipo de riesgo cambiario conocido desde el principio. El problema estructural no fue la falta de sofisticación financiera del equipo —el análisis de costos original era técnicamente riguroso—. Fue el diseño del plan como una secuencia lineal sin puntos de revisión motivados, bajo el supuesto implícito de que el mundo permanecería razonablemente quieto durante los tres meses de ejecución.

Aplícalo a tu proyecto

| “Mientras decides, el mundo sigue decidiendo por su cuenta.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Cuando dos historias igual de lógicas te llevan a conclusiones opuestas, el problema no es la lógica. Es el framing.””

El Yale Shooting Problem, llevado a negocio

“Cuando dos historias igual de lógicas te llevan a conclusiones opuestas, el problema no es la lógica. Es el framing.”

En 1986, Steve Hanks y Drew McDermott publicaron un contraejemplo que sacudió, de forma genuina y bien documentada, a la comunidad de inteligencia artificial dedicada al razonamiento temporal (Hanks & McDermott, 1986). El contraejemplo, hoy conocido en toda la literatura del área como el Yale Shooting Problem, tiene una estructura deliberadamente simple. Un arma se carga en el instante 1. No ocurre ninguna otra acción explícitamente declarada hasta el instante 5, cuando el arma se dispara hacia una persona, a la que llamaremos Fred. Bajo cualquier intuición de sentido común razonable, Fred debería estar muerto en el instante 6.

Sin embargo, cuando esta historia se formalizó usando la solución de McCarthy vigente en ese momento —basada en circunscripción y en minimizar la cantidad de comportamiento "anormal" respecto a un principio de inercia general (McCarthy, 1980, 1986)—, la lógica resultante permitía, con la misma validez formal, una segunda conclusión: que el arma se había descargado sola, sin ninguna causa explícita, durante el periodo de espera entre el instante 1 y el instante 5, y que Fred, por lo tanto, seguía vivo después del disparo. Ambas historias —la esperada, donde Fred muere, y la inesperada y en apariencia absurda, donde el arma se descarga misteriosamente— resultaban ser modelos igualmente válidos y consistentes con los mismos hechos declarados en la teoría formal.

El problema no era que la lógica de McCarthy estuviera simplemente "mal" en un sentido técnico burdo. El problema, más sutil y más interesante, era que, sin una noción explícita y formalizada de causalidad, ambas historias resultaban ser igualmente consistentes con

el conjunto de hechos declarados: la lógica de minimización, tal como estaba formulada entonces, no tenía ningún mecanismo interno para preferir sistemáticamente la historia donde "las cosas persisten hasta que algo específico las cambia" sobre la historia, igualmente consistente formalmente, donde "algo cambió sin que existiera ninguna causa identificada para ese cambio".

Esta anécdota, hoy célebre en toda la literatura de inteligencia artificial sobre razonamiento temporal como el Yale Shooting Problem, tiene un equivalente estructural casi exacto en cualquier comité directivo que hayas presenciado: dos personas presentan interpretaciones igualmente lógicas de exactamente los mismos datos disponibles, y llegan a conclusiones diametralmente opuestas, porque cada una está aplicando, sin declararlo explícitamente, un supuesto de persistencia distinto sobre qué se mantiene igual y qué requiere una causa explícita identificada para poder considerarse que cambió. Reconocer esta estructura formal —dos historias igualmente válidas derivadas del mismo conjunto de hechos— es, con frecuencia, el primer paso genuinamente productivo para resolver el desacuerdo: no se trata, casi nunca, de que una de las dos partes tenga acceso a mejores datos que la otra. Se trata de que cada una está usando un framing distinto e implícito para interpretar los mismos datos compartidos.

La lección práctica inmediata: cuando dos personas de tu equipo disienten de forma persistente sobre una proyección construida a partir de exactamente los mismos números, antes de pedir "más datos" —una respuesta reflexiva pero, en este tipo de desacuerdo específico, casi siempre ineficaz—, pregunta de forma explícita a cada una: ¿qué estás asumiendo que se mantiene igual, y qué estás asumiendo que cambió sin que lo hayamos observado directamente? En una proporción sorprendentemente alta de los casos que he observado en contextos de analítica y estrategia de datos, ahí es exactamente donde está el verdadero desacuerdo, no en la calidad o cantidad de la evidencia disponible.

Dos lecturas igualmente lógicas del mismo número

En el comité mensual de resultados de una cadena regional de farmacias, se presentó un dato que generó un desacuerdo prolongado y, en su momento, genuinamente difícil de resolver: las ventas de una categoría específica —productos de cuidado personal de precio medio— habían caído ocho por ciento respecto al mes anterior, una variación considerablemente por encima del ruido estadístico normal de esa categoría.

El analista senior de la categoría, con acceso a los mismos datos que el resto del comité, presentó una interpretación: la caída reflejaba una contracción genuina de la demanda del consumidor en ese segmento de precio, posiblemente relacionada con presión inflacionaria general sobre el ingreso disponible de la base de clientes objetivo de esa categoría. Su recomendación, coherente con esa lectura, era ajustar el forecast de la categoría hacia abajo para los próximos meses y evaluar una estrategia de precio más agresiva para estimular demanda.

El gerente de operaciones de tienda, presente en el mismo comité y con acceso a los mismos números agregados, ofreció una lectura completamente distinta: sospechaba un problema de quiebre de inventario no capturado correctamente por el sistema central de reportería, posiblemente originado en un cambio reciente en el proceso de reposición para esa categoría específica, implementado por el área de cadena de suministro semanas antes sin que el área comercial fuera plenamente consciente del cambio. Su recomendación era investigar el nivel de inventario disponible por tienda antes de tomar cualquier decisión sobre forecast o precio.

Ambas lecturas eran perfectamente consistentes, en un sentido lógico estricto, con el único dato disponible en la sala: la caída de ocho por ciento en ventas. La discusión, que inicialmente se centró en defender cada posición con el mismo número una y otra vez —un patrón de desacuerdo notoriamente improductivo—, cambió de calidad en el

momento en que alguien, aplicando explícitamente la lógica de este capítulo, preguntó: ¿qué causa concreta y verificable respalda cada una de las dos hipótesis, más allá de la plausibilidad general de la historia que cada quien está contando? Esa pregunta redirigió la conversación de un debate sobre interpretaciones hacia una tarea de verificación concreta: revisar el nivel de inventario disponible por tienda para esa categoría específica en las semanas relevantes. La verificación, realizada en menos de un día, confirmó la segunda hipótesis: un problema de reposición en aproximadamente el veinte por ciento de las tiendas de la cadena, no capturado correctamente por el sistema de alertas de quiebre existente, que sí había afectado ventas, pero por una causa completamente distinta —y mucho más rápida de corregir— que una contracción genuina de demanda del consumidor.

EJERCICIO 6 — EL YALE SHOOTING PROBLEM EN TU COMITÉ

1. La próxima vez que dos colegas disientan sobre una misma cifra, pide a cada uno que declare su supuesto de persistencia por default.
2. Pregunta qué causa concreta respalda la versión de "algo cambió".
3. Documenta cuál hipótesis tenía evidencia causal explícita y cuál dependía solo de plausibilidad.

Aplicalo a tu proyecto

“Cuando dos historias igual de lógicas te llevan a conclusiones opuestas, el problema no es la lógica. Es el framing.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Nixon era cuáquero y era republicano. Un hecho sugiere que era pacifista. El otro, que no. Los dos son ciertos.””

Cuando dos supuestos válidos chocan: el diamante de Nixon

"Nixon era cuáquero y era republicano. Un hecho sugiere que era pacifista. El otro, que no. Los dos son ciertos."

Antes del Yale Shooting Problem, la lógica no monotónica ya conocía un problema hermano, formulado en el contexto de los sistemas de razonamiento por default: el problema de extensiones múltiples, popularizado en la literatura mediante el ejemplo del "diamante de Nixon" (Reiter & Criscuolo, 1981). El ejemplo, en su forma clásica: los cuáqueros suelen ser, por default, pacifistas. Los republicanos, en el ejemplo original formulado en el contexto político estadounidense de los años setenta, suelen ser, por default, no pacifistas. Richard Nixon era, simultáneamente, cuáquero y republicano. ¿Qué default gana? La lógica formal, por sí sola, no tiene ningún mecanismo interno para decidirlo de forma no arbitraria —y, de hecho, en un número considerable de casos análogos, no debería intentar decidirlo de forma forzada: hay situaciones donde la conclusión correcta y honesta es reconocer explícitamente la incertidumbre, no forzar una respuesta con apariencia de certeza.

La distinción crítica —y la que, en mi experiencia observando equipos de datos y estrategia, la mayoría no hace de forma deliberada— es entre dos tipos de desacuerdo que superficialmente se ven idénticos: desacuerdos donde genuinamente no existe suficiente información disponible para decidir con confianza razonable, y desacuerdos donde sí existe esa información, pero el framing usado por cada una de las partes involucradas está ocultando esa información relevante en lugar de exponerla. El primer tipo de desacuerdo se resuelve con humildad epistémica explícita: "no lo sabemos todavía con la confianza necesaria, y aquí está el rango razonable de posibilidades". El segundo tipo se resuelve exponiendo el framing específico de cada parte hasta identificar con precisión el punto exacto donde ambos framings divergen.

En la práctica organizacional, este patrón del diamante de Nixon aparece con notable regularidad cada vez que dos áreas distintas de una misma empresa usan reglas de default diferentes para el mismo tipo de decisión, sin que ninguna de las dos áreas sea consciente de que la otra opera bajo un default distinto. Un ejemplo recurrente: el área de Finanzas puede operar bajo el default implícito de que "un contrato sin renovación explícitamente documentada se considera terminado al vencer su plazo original", mientras que el área Comercial opera bajo el default implícito de que "un contrato sin cancelación explícitamente documentada se considera vigente por default, incluso después de vencido su plazo original, hasta que alguna de las partes lo cancele formalmente". Ninguno de los dos defaults es, por sí solo, absurdo o irracional; de hecho, ambos son razonables dentro de la lógica particular de cada área. El problema aparece, de forma silenciosa y potencialmente costosa, cuando nadie ha hecho explícito que existen dos defaults distintos operando simultáneamente sobre exactamente la misma realidad contractual.

La herramienta operativa correspondiente es relativamente simple de nombrar, aunque considerablemente más difícil de practicar de forma consistente en el día a día organizacional: cuando detectes un diamante de Nixon organizacional —dos defaults razonables, aplicados por dos partes distintas, en conflicto directo entre sí sobre la misma realidad— no elijas uno de los dos de forma arbitraria, y tampoco intentes promediar entre ambos como si fuera una solución de compromiso válida. En su lugar, haz explícita una regla de prioridad clara entre los dos defaults en conflicto, documéntala formalmente, y comunícala activamente a ambas áreas involucradas. El costo de comunicar mal una regla de prioridad es, de forma consistente, considerablemente menor que el costo acumulado de que dos áreas sigan operando indefinidamente con supuestos incompatibles entre sí, sin saberlo.

EJEMPLO EXTENDIDO

El diamante de Nixon en la renovación de contratos

Una empresa de servicios profesionales con un modelo de negocio basado en contratos anuales recurrentes con clientes corporativos descubrió, durante una auditoría interna de ingresos rutinaria, una discrepancia sistemática entre el reporte de ingresos recurrentes generado por el área Comercial y el reporte de ingresos generado por el área de Finanzas, para el mismo conjunto de clientes, en el mismo periodo.

La investigación de la discrepancia reveló exactamente el patrón de diamante de Nixon descrito en este capítulo. El sistema de gestión de relaciones con clientes (CRM), operado principalmente por el área Comercial, había sido configurado, años atrás, por un analista que ya no trabajaba en la empresa, bajo el principio de que un contrato permanecía como "activo" en el sistema por default, salvo que existiera una cancelación explícitamente registrada por el cliente o por el equipo de cuenta. El sistema de facturación y reconocimiento de ingresos, operado por el área de Finanzas y configurado de forma completamente independiente por un equipo distinto, había sido diseñado bajo el principio opuesto: un contrato se consideraba terminado por default al vencer su plazo original, salvo que existiera una renovación explícitamente firmada y registrada en el sistema.

Durante varios años, ambos sistemas habían coexistido sin generar un conflicto visible, porque en la gran mayoría de los casos reales, los clientes efectivamente renovaban de forma explícita o cancelaban de forma explícita dentro de una ventana de tiempo razonable, haciendo que la diferencia entre ambos defaults casi nunca se manifestara de forma práctica. El problema se hizo visible únicamente cuando, tras un cambio en el proceso comercial que introdujo mayor ambigüedad en el momento exacto de renovación de un subconjunto de contratos, un número creciente de casos empezó a caer exactamente en la zona gris donde los dos defaults divergían: contratos vencidos, sin cancelación explícita registrada, pero también sin renovación explícita formalmente firmada todavía, en negociación.

El área Comercial, bajo su propio default, seguía reportando esos

contratos como ingresos recurrentes activos en sus proyecciones internas. El área de Finanzas, bajo el default opuesto, ya no los reconocía como ingreso en sus estados financieros formales. La discrepancia acumulada, al momento de la auditoría, representaba una diferencia material entre el ingreso recurrente proyectado internamente por Comercial y el ingreso efectivamente reconocido por Finanzas, generando confusión considerable en el proceso de planeación financiera y, en al menos un caso documentado, una proyección de ingresos presentada a un inversionista que tuvo que corregirse posteriormente. La resolución del problema no requirió decidir cuál de los dos defaults era "el correcto" en abstracto —ambos eran defensibles dentro de la lógica de su propia área—. Requirió que ambas áreas acordaran explícitamente una única regla de prioridad, documentada formalmente y aplicada de manera consistente en ambos sistemas: un contrato se considera activo solo si existe evidencia explícita de renovación firmada, o si se encuentra dentro de una ventana de gracia predefinida y acotada después del vencimiento, pasada la cual se reclasifica automáticamente como inactivo salvo renovación formal.

EJERCICIO 7 — MAPA DE DEFAULTS EN CONFLICTO

1. Identifica dos áreas de tu organización que interactúan sobre el mismo tipo de dato o decisión.
2. Pregunta a cada una, por separado, cuál es su regla "por default" para el caso ambiguo más común entre ellas.
3. Si las reglas difieren, documenta una regla de prioridad explícita y compártela con ambas áreas.

Aplicalo a tu proyecto

“Nixon era cuáquero y era republicano. Un hecho sugiere que era pacifista. El otro, que no. Los dos son ciertos.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““No es que el periódico tape el ducto. Es que el cuarto se vuelve sofocante, y nadie escribió esa regla.””

Ramificaciones: el efecto dominó que nadie modeló

“No es que el periódico tape el ducto. Es que el cuarto se vuelve sofocante, y nadie escribió esa regla.”

Existe un tipo de cambio estructuralmente distinto a los que hemos discutido hasta este punto del libro: no se trata de que algo deje de mantenerse igual por sí solo, de forma espontánea y sin causa, sino de que un cambio directo, deliberadamente causado por una acción concreta, dispara en cascada otros cambios adicionales que nadie causó de forma directa. El ejemplo clásico de la literatura académica, discutido por Ginsberg y Smith (1988): si colocas un periódico sobre el único ducto de ventilación de un cuarto, no solo el ducto queda cubierto —ese es el efecto directo, deliberadamente causado por la acción de colocar el periódico—. El cuarto, además, se vuelve progresivamente sofocante por falta de circulación de aire, y las páginas del periódico podrían agitarse ligeramente si el flujo de aire residual es suficientemente fuerte para moverlas antes de detenerse por completo. Nadie “causó” directamente, con una acción explícita, que el cuarto se pusiera sofocante; fue una consecuencia derivada, en cascada, de haber tapado el ducto. A este fenómeno específico, la literatura de inteligencia artificial lo denomina el problema de las ramificaciones (the ramification problem) (Finger, 1988).

Intentar escribir explícitamente cada efecto derivado posible es, una vez más, una explosión combinatoria estructuralmente idéntica a la del frame problem original: el cuarto se vuelve sofocante solo si ese ducto era, efectivamente, el único disponible; solo si hay alguien presente adentro del cuarto en ese momento para experimentar la falta de ventilación como un problema relevante; solo si el periódico efectivamente logra bloquear el flujo de aire de forma completa y no parcial. La alternativa ingenua de escribir explícitamente cada condición

encadenada de este tipo convierte lo que originalmente era, en parte, un problema computacional de inferencia derivada, en un problema puramente representacional igual de intratable que el frame problem original, porque ahora hay que enumerar exhaustivamente todas las condiciones bajo las cuales cada efecto derivado específico efectivamente ocurre.

En el contexto de negocio, las ramificaciones son precisamente la razón estructural por la que los proyectos que en apariencia son "simples" y contenidos terminan generando consecuencias no anticipadas en áreas de la organización que, en el momento del diseño original, nadie conectó de forma explícita con el cambio propuesto. Cambiar una política de devoluciones al consumidor no afecta únicamente, de forma directa, al área de Servicio al Cliente que la administra en el día a día. Puede disparar, en cascada, cambios en la proyección de flujo de caja, en el inventario que el sistema reporta como efectivamente disponible para venta, y en los incentivos de la fuerza de ventas, si su esquema de comisiones está calculado sobre ventas netas de devoluciones. Nadie "causó" directamente, con una acción explícita, ninguno de esos tres efectos secundarios; se derivaron de una cadena de dependencias estructurales que ya existía en la organización antes del cambio de política, pero que nadie había hecho suficientemente visible para el equipo que propuso y aprobó el cambio original.

La defensa práctica genuinamente efectiva contra el problema de las ramificaciones no consiste en intentar enumerarlas exhaustivamente de antemano —esa estrategia es, como acabamos de discutir, computacionalmente imposible en la práctica y, si se intenta de forma rigurosa, conduce de vuelta directamente a la trampa combinatoria descrita en el Capítulo 7—. La defensa consiste, en cambio, en mapear de forma sistemática, para cualquier cambio de suficiente significancia dentro de la organización, las dependencias de segundo orden más probables antes de implementar el cambio, usando específicamente a las personas que operan de forma directa y cotidiana cada uno de los sistemas conectados, no exclusivamente al equipo que originalmente propone el cambio y que, casi por definición, tiene visibilidad limitada

sobre las dependencias que existen fuera de su propio ámbito directo de responsabilidad.

EJEMPLO EXTENDIDO

El cambio de umbral que movió cinco sistemas distintos

Un equipo de marketing de una empresa de suscripción digital propuso, como parte de una iniciativa de mejora de retención de clientes, un cambio en apariencia sencillo: modificar el umbral de días de inactividad que el sistema usaba para clasificar a un cliente como "inactivo" dentro del CRM, reduciéndolo de noventa a sesenta días, con el argumento de que una definición más sensible permitiría detectar señales tempranas de riesgo de cancelación y activar campañas de reactivación con mayor anticipación.

El cambio, propuesto y aprobado inicialmente dentro del propio equipo de marketing sin una consulta formal y sistemática a otras áreas, se implementó como una modificación de un único parámetro dentro del sistema de CRM: el umbral numérico de días. Desde la perspectiva estrictamente técnica del equipo que implementó el cambio, se trataba de una modificación mínima, de bajo riesgo aparente, y contenida enteramente dentro de su propio sistema.

En las semanas siguientes a la implementación, sin embargo, el cambio generó una cascada de efectos derivados en al menos cuatro sistemas adicionales, ninguno de los cuales había sido consultado antes del cambio. El cálculo de la tasa de churn mensual, un indicador clave reportado de forma rutinaria al comité directivo, cambió de forma abrupta simplemente porque la definición subyacente de "cliente inactivo" había cambiado, generando una discontinuidad en la serie histórica que inicialmente se interpretó, erróneamente, como un deterioro genuino en la retención de clientes, antes de que alguien identificara la causa metodológica real. Los segmentos automatizados de campañas de marketing, contruidos sobre la misma definición de

actividad, cambiaron de tamaño y composición de forma significativa, alterando el comportamiento de campañas automatizadas ya en producción sin que nadie hubiera anticipado ni validado ese efecto. El modelo de forecast de ingresos recurrentes, operado por el área de Finanzas y que dependía de la misma clasificación de actividad como uno de sus insumos, generó proyecciones desalineadas con el forecast del trimestre anterior, generando preguntas difíciles de responder en la siguiente reunión de planeación financiera. Y el sistema de alertas de riesgo de cancelación, operado por el equipo de éxito del cliente (customer success), empezó a generar un volumen de alertas considerablemente mayor al histórico, saturando la capacidad operativa del equipo encargado de darles seguimiento manual.

Ninguno de estos cuatro efectos había sido causado de forma directa por el cambio original —el equipo de marketing no modificó, en ningún momento, el cálculo de churn, la lógica de segmentación de campañas, el modelo de forecast financiero, ni el sistema de alertas de éxito del cliente—. Todos esos efectos se derivaron, en cascada, de una dependencia estructural preexistente: los cuatro sistemas consumían, de una forma u otra, la misma variable de "estatus de actividad del cliente" que el equipo de marketing había modificado sin ser plenamente consciente de cuántos otros sistemas dependían de esa misma definición compartida. La remediación, una vez identificado el problema completo, requirió no solo ajustar los cuatro sistemas afectados para que reflejaran de forma coherente la nueva definición, sino también establecer, hacia adelante, un proceso formal de revisión de impacto cruzado antes de modificar cualquier definición de datos compartida entre múltiples áreas —exactamente el tipo de mapeo de ramificaciones que este capítulo recomienda como práctica preventiva.

EJERCICIO 8 — MAPA DE RAMIFICACIONES

1. Antes del próximo cambio de política o de umbral en un sistema clave, dibuja un mapa simple: el cambio en el centro, y una flecha hacia cada sistema o KPI que dependa de ese dato.
2. Para cada flecha, escribe si el efecto es directo o derivado.
3. Comparte el mapa con los dueños de cada sistema conectado antes de implementar.

Aplicalo a tu proyecto

“No es que el periódico tape el ducto. Es que el cuarto se vuelve sofocante, y nadie escribió esa regla.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“La ausencia de una alarma no es evidencia de que todo está bien. Es evidencia de que nadie construyó la alarma.”

La falacia de "si no cambió, sigue igual"

"La ausencia de una alarma no es evidencia de que todo está bien. Es evidencia de que nadie construyó la alarma."

Hay una trampa cognitiva y estructural que subyace a buena parte de los fracasos de framing discutidos hasta este punto del libro, y merece un capítulo dedicado exclusivamente a ella porque es, con bastante probabilidad, la más peligrosa de todas las que veremos: confundir sistemáticamente "no tengo evidencia disponible de que esto haya cambiado" con "sé, con confianza justificada, que esto no ha cambiado". Son afirmaciones epistemológicamente distintas de forma radical, y sin embargo los equipos las tratan, en la práctica cotidiana, como intercambiables con una frecuencia notable, particularmente bajo presión de tiempo o cuando la carga cognitiva de mantener la distinción resulta incómoda.

Esta confusión tiene una explicación estructural, no meramente una explicación de descuido individual: los sistemas de monitoreo, por diseño y casi por definición, únicamente te avisan de aquello que alguien decidió, en algún momento, específicamente vigilar. Si nadie construyó una alerta diseñada para un supuesto particular, el silencio de ese sistema de monitoreo no significa, en ningún sentido lógico riguroso, que el supuesto en cuestión siga siendo válido. Significa, de forma literal y precisa, que nadie preguntó activamente por ese supuesto específico. Y sin embargo, interpretamos habitualmente el silencio de un dashboard o de un sistema de alertas como una forma implícita de confirmación positiva, porque psicológicamente resulta considerablemente más cómodo operar bajo esa interpretación que sostener activamente la incertidumbre genuina.

La distinción técnica que resulta útil aquí, tomada de forma directa de las soluciones formales al frame problem que desarrollaremos con mayor

detalle en la Parte III, es la diferencia entre persistencia por default —asumo, como una apuesta consciente y explícitamente reconocida como tal, que algo sigue igual mientras no tenga evidencia específica de lo contrario, sabiendo plenamente que se trata de una apuesta razonable, no de un hecho verificado— y persistencia verificada —confirmé de forma activa y reciente que algo, efectivamente, sigue igual—. La primera categoría es absolutamente necesaria para poder actuar en el mundo sin caer en parálisis total por análisis; ningún sistema, humano ni artificial, tiene la capacidad de verificar exhaustivamente cada supuesto relevante antes de cada decisión que necesita tomar. Pero tratar sistemáticamente la primera categoría como si perteneciera a la segunda —presentar una persistencia meramente asumida con el mismo grado de confianza retórica que una persistencia efectivamente verificada— es, precisamente, la falacia específica que este capítulo busca nombrar con precisión suficiente para que puedas reconocerla en tiempo real.

La práctica correctiva concreta es, en su formulación, sorprendentemente simple de instalar como hábito de equipo: cuando presentes un supuesto relevante dentro de una decisión de cierta importancia, etiquétalo explícitamente, en el documento o en la presentación misma, como "verificado el [fecha específica]" o como "asumido por default, sin verificación reciente registrada". Esa sola etiqueta, casi trivial de escribir en términos de esfuerzo requerido, cambia de forma sustancial cómo el resto del equipo y, más importante todavía, cómo un comité directivo evalúa el nivel de riesgo real asociado a una decisión que depende de ese supuesto específico.

EJEMPLO EXTENDIDO

El inventario "confirmado" que nunca se verificó

Una cadena de tiendas departamentales construyó, con varios meses de anticipación, su plan de compras y su forecast de ventas para la temporada alta de fin de año, un periodo que representaba, de forma

consistente año tras año, una proporción muy significativa de los ingresos anuales totales de la empresa. Una parte central de ese plan asumía disponibilidad suficiente de inventario en el centro de distribución principal para las categorías con mayor rotación esperada durante la temporada.

El equipo de planeación, al presentar el plan al comité directivo, afirmó que el nivel de inventario disponible estaba "confirmado" y era suficiente para sostener las proyecciones de venta planeadas. Esa afirmación, sin embargo, se basaba enteramente en la ausencia de alertas activas en el sistema central de gestión de inventario, no en una verificación activa y reciente del nivel físico real de inventario disponible en el centro de distribución. La alerta de inventario del sistema, diseñada originalmente varios años atrás, estaba configurada exclusivamente para detectar faltantes —niveles de inventario por debajo de un umbral mínimo predefinido—; nunca había sido diseñada ni configurada para detectar excesos de inventario mal etiquetados como disponibles debido a errores de reconciliación, un tipo de problema distinto que el sistema simplemente no estaba diseñado para capturar.

Semanas antes del inicio de la temporada alta, una devolución masiva de un pedido corporativo de gran volumen —cancelado por un cliente empresarial que había ordenado inventario para un evento propio que finalmente no se realizó— generó una entrada de inventario en el sistema que, debido a un error de proceso en el área de logística inversa, quedó etiquetada incorrectamente como "disponible para venta regular" cuando, en realidad, una parte sustancial de ese inventario devuelto presentaba condiciones de empaque y calidad que lo hacían inadecuado para venta directa al consumidor final sin un proceso de reacondicionamiento previo que nunca se ejecutó a tiempo.

El sistema de alertas, fiel a su diseño original limitado a detectar faltantes, nunca generó ninguna señal sobre esta discrepancia, porque desde su perspectiva estrictamente numérica, el nivel de inventario disponible efectivamente cumplía con el umbral mínimo requerido —el problema no era una cantidad insuficiente de unidades registradas, sino

la calidad y disponibilidad real de una parte de esas unidades—. El equipo de planeación, al no recibir ninguna alerta, interpretó ese silencio como confirmación de que el plan seguía siendo válido, sin realizar ninguna verificación física adicional antes del inicio de la temporada. La discrepancia se descubrió únicamente durante los primeros días de la temporada alta, cuando varias tiendas reportaron faltantes inesperados en categorías que, según el sistema central, deberían haber estado plenamente abastecidas, generando una crisis operativa de último momento en el peor momento posible del año para gestionarla, precisamente cuando la capacidad operativa del equipo de logística estaba ya al límite por el volumen normal de la temporada.

EJERCICIO 9 — ETIQUETA TUS SUPUESTOS

1. Toma la última presentación o reporte importante de tu equipo.
2. Marca cada supuesto clave como "verificado" o "asumido por default".
3. Si más de la mitad son "asumido por default" en decisiones de alto impacto, prioriza verificar los de mayor costo de ruptura.

Aplicalo a tu proyecto

“La ausencia de una alarma no es evidencia de que todo está bien. Es evidencia de que nadie construyó la alarma.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

CIERRE DE PARTE II

Antes de continuar

¿Cuál de los patrones de fragilidad —exceso de reglas, concurrencia ignorada, ramificaciones, la falacia del silencio— reconoces con más frecuencia en tu organización?

TU RESPUESTA

UNA ACCIÓN CONCRETA PARA ESTA SEMANA

El Toolkit

Herramientas de framing

“No necesitas más reglas. Necesitas mejor framing por default.”

“La regla más poderosa de la lógica no monotónica cabe en una frase: las cosas siguen igual, a menos que algo, específicamente, las cambie.”

Persistencia por default: qué asumir cuando nada más se sabe

“La regla más poderosa de la lógica no monotónica cabe en una frase: las cosas siguen igual, a menos que algo, específicamente, las cambie.”

Llegamos a la primera herramienta de solución formal seria dentro del Toolkit de este libro. John McCarthy propuso formalizar el sentido común mediante un único principio general y compacto, en lugar de miles de axiomas particulares enumerados uno por uno: si una propiedad determinada es cierta en una situación dada, y la acción que ocurre no es explícitamente "anormal" respecto a esa propiedad específica, entonces la propiedad en cuestión sigue siendo cierta en la situación resultante de ejecutar esa acción. Este es, formalmente, el principio de inercia (McCarthy, 1980, 1986): por default, las cosas persisten a través del tiempo, salvo una excepción explícita y documentada.

En notación formal simplificada, siguiendo la reformulación de Hanks y McDermott (1986) del planteamiento original de McCarthy, este principio se expresa como: para todo fluent f , evento e y situación s , si el fluent se cumple en la situación s , y el evento e no es anormal respecto al fluent f en la situación s , entonces el fluent f se sigue cumpliendo en la situación resultante de ejecutar el evento e sobre s . La elegancia de este único principio general es que reemplaza, de un solo golpe formal, la necesidad de escribir un axioma de framing separado para cada combinación específica de acción y propiedad estable que existiría bajo el enfoque puramente monotónico descrito en el Capítulo 7.

Traducido al lenguaje de diseño de sistemas de negocio, el principio de persistencia por default es la base conceptual de cualquier sistema de valores por default bien diseñado: en lugar de que cada proceso o sistema declare explícitamente, de forma exhaustiva, qué propiedades NO cambian con cada acción —una lista, como ya vimos, potencialmente

interminable—, el sistema declara explícitamente únicamente qué SÍ cambia con cada acción específica, y asume, por default y de forma estructural, que todo lo demás persiste sin necesidad de declaración adicional. Es exactamente el mismo principio de diseño que subyace a los formularios digitales bien diseñados, que solo solicitan al usuario la información que efectivamente cambió desde la última actualización registrada, en lugar de solicitar de nuevo, cada vez, la totalidad de la información previamente proporcionada.

McCarthy mismo reconoció que este principio, aplicado de forma ingenua dentro de la situation calculus original, tenía una limitación seria que se manifestaría de forma dramática apenas unos años después de su formulación, con la publicación del Yale Shooting Problem que ya discutimos en el Capítulo 9: la noción de "anormalidad", tal como estaba formulada originalmente, no incluía ningún requisito explícito de causalidad, lo cual permitía, en ciertos casos formalmente construibles, que el principio de persistencia generara conclusiones tan válidas formalmente como absurdas desde el sentido común. Esta limitación, lejos de invalidar el principio completo, motivó buena parte del desarrollo posterior de la lógica no monotónica temporal que estudiaremos en los capítulos siguientes de esta parte del libro.

La disciplina práctica concreta que se deriva de este capítulo: para cualquier proceso o sistema que estés diseñando o rediseñando desde hoy, en lugar de preguntar reflexivamente "¿qué necesito declarar de forma explícita que se mantiene igual?", pregunta en cambio "¿qué necesito declarar de forma explícita que cambia con esta acción específica, y qué condición de anormalidad tendría que ocurrir para que algo adicional, inesperadamente, también cambiara como consecuencia?". Este giro deliberado en la formulación de la pregunta — de enumerar exhaustivamente lo estable a enumerar únicamente lo que efectivamente cambia junto con sus condiciones específicas de anormalidad— es, por sí solo, probablemente la reducción de complejidad más poderosa y de más fácil aplicación inmediata que puedes incorporar a cualquier sistema que diseñes a partir de hoy.

El formulario que dejó de pedir lo mismo dos veces

Un proveedor de software empresarial para el sector de seguros identificó, a través de análisis de comportamiento de usuario, una tasa de abandono considerablemente alta en su proceso anual de renovación de pólizas para clientes existentes: un porcentaje significativo de usuarios que iniciaban el proceso de renovación no llegaban a completarlo, generando trabajo manual adicional para el equipo de servicio al cliente que debía dar seguimiento telefónico a esos casos incompletos.

El diagnóstico inicial, realizado con apoyo de investigación de experiencia de usuario, reveló una causa raíz directamente relacionada con el diseño original del formulario de renovación: cada año, al iniciar el proceso, el sistema solicitaba al cliente que volviera a proporcionar, desde cero, la totalidad de la información relevante para la póliza — datos personales, información de los bienes asegurados, historial de siniestros, beneficiarios—, exactamente como si se tratara de una contratación completamente nueva, sin ningún reconocimiento explícito de que esa misma información ya existía, verificada y almacenada, en el sistema desde la contratación o renovación del año anterior.

El equipo de producto, al aplicar de forma deliberada el principio de persistencia por default descrito en este capítulo, rediseñó por completo el proceso de renovación bajo una lógica distinta: en lugar de solicitar toda la información desde cero, el sistema presentaba al usuario la información previamente registrada, ya prellenada, y solicitaba explícitamente únicamente la confirmación de que esa información seguía siendo correcta, o la actualización específica de únicamente aquellos campos que efectivamente habían cambiado desde la última renovación —un cambio de domicilio, un vehículo adicional, un beneficiario modificado—. El diseño técnico subyacente aplicaba, en esencia, el mismo principio formal descrito por McCarthy: asumir persistencia por default de cada campo, y solicitar activamente solo la

información correspondiente a un cambio explícito.

El resultado, medido en el ciclo de renovación siguiente al rediseño, fue una reducción de la tasa de abandono del proceso de renovación del orden del treinta y cuatro por ciento, junto con una reducción correspondiente en la carga de trabajo del equipo de servicio al cliente dedicado a dar seguimiento a renovaciones incompletas. Un beneficio adicional, inicialmente no anticipado por el equipo de producto, fue una mejora medible en la calidad de los datos almacenados: al pedir confirmación explícita en lugar de recaptura completa, el sistema redujo también los errores de tipeo y las inconsistencias entre el registro del año anterior y el nuevo registro, un problema que, bajo el diseño original, generaba trabajo adicional de reconciliación de datos para el equipo de operaciones.

EJERCICIO 10 — REDISEÑA CON PERSISTENCIA POR DEFAULT

1. Elige un formulario, proceso o reporte recurrente en tu equipo.
2. Identifica qué campos se vuelven a pedir o recalcular cada vez, aunque casi nunca cambien.
3. Rediseña el proceso para que solo pida explícitamente lo que cambió, asumiendo persistencia del resto por default.

Aplícalo a tu proyecto

“La regla más poderosa de la lógica no monotónica cabe en una frase: las cosas siguen igual, a menos que algo, específicamente, las cambie.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“Un fluent solo cambia si un evento específico lo cambia. Si el evento no ocurrió, el fluent no cambió. Punto.”

Cierre por explicación: solo cambia lo que tiene una causa

“Un fluent solo cambia si un evento específico lo cambia. Si el evento no ocurrió, el fluent no cambió. Punto.”

Una variante importante del principio de persistencia por default, propuesta de forma independiente por Ernest Davis (1990) y Lenhart Schubert (1990), ataca el frame problem desde un ángulo formalmente distinto, aunque relacionado: en lugar de establecer "todo persiste salvo anormalidad explícita", este enfoque establece "una propiedad determinada solo puede cambiar de valor si ocurre uno de los eventos específicos previamente identificados como causa legítima de ese cambio; si ninguno de esos eventos específicos ocurrió, la propiedad no cambió, sin necesidad de ninguna otra justificación adicional". A esta idea se le denomina, en la literatura formal, cierre por explicación (explanation closure), precisamente porque cierra de forma exhaustiva, para cada propiedad relevante del sistema, el conjunto completo de causas legítimas posibles de cambio.

La diferencia con el principio de persistencia general del capítulo anterior parece sutil en una primera lectura, pero resulta considerablemente poderosa en aplicación práctica: el cierre por explicación te obliga, como parte del propio proceso de diseño del sistema, a enumerar de antemano todas las causas legítimas de cambio para cada dato de suficiente importancia dentro de tu operación. Si, posteriormente, alguien reporta un cambio en ese dato que no corresponde a ninguna de las causas previamente enumeradas en la lista cerrada, sabes con certeza formal —no meramente con sospecha— que algo anómalo está ocurriendo: un error de captura, un proceso no autorizado ejecutándose fuera del flujo esperado, o, en casos más graves, un intento de fraude. Y sabes esto sin necesidad de investigar cada caso individual desde cero, porque el propio diseño del sistema ya

incorpora, de forma estructural, el criterio de detección.

Esta lógica formal es, en esencia, la que subyace a los sistemas de auditoría y de control de cambios bien diseñados en cualquier organización madura: un campo determinado, por ejemplo "estatus de un contrato", debería poder cambiar de valor únicamente mediante un conjunto acotado y explícito de eventos autorizados —firma inicial, proceso formal de renovación, cancelación formalmente documentada—; si el campo cambia de valor sin que ninguno de esos eventos específicos quede registrado de forma correspondiente en el sistema de auditoría, el sistema debería, idealmente, marcar automáticamente ese cambio como una anomalía que requiere revisión, en lugar de depender de que un ser humano lo detecte por casualidad durante una revisión manual periódica, que puede tardar meses en ocurrir o simplemente no ocurrir nunca.

Es importante notar, siguiendo el análisis de Reiter (1991), que el cierre por explicación, aplicado de forma ingenua y sin optimización adicional, puede requerir un número de axiomas del orden de dos veces el número de fluents relevantes del sistema —uno para las condiciones que hacen que el fluent se vuelva verdadero, y otro para las condiciones que hacen que se vuelva falso—, a menos que se mantenga una distinción cuidadosa entre eventos primitivos y eventos derivados, en cuyo caso Reiter demostró que el número de axiomas necesarios puede reducirse a un orden proporcional a la suma del número de fluents más el número de acciones, una mejora computacional significativa sobre el enfoque puramente monotónico original.

La disciplina operativa concreta que se deriva de este capítulo consiste en construir, para cada dato crítico y sensible de tu operación, una lista corta, explícita y deliberadamente exhaustiva de "las únicas causas legítimas de cambio" para ese dato. Es importante notar que esta no es una lista de lo que NO cambia —eso ya está cubierto, de forma más general, por el principio de persistencia por default discutido en el capítulo anterior—. Es, específicamente, una lista acotada de las causas válidas y autorizadas de cambio. Cualquier cambio detectado que caiga

fuera de esa lista predefinida es, por la propia construcción lógica del sistema, una señal que merece atención inmediata y prioritaria, sin necesidad de un juicio de valor caso por caso.

EJEMPLO EXTENDIDO

El campo que solo debía cambiar por tres razones

Una institución financiera de tamaño mediano, tras un incidente de seguridad relativamente menor pero suficientemente serio como para motivar una revisión completa de sus controles internos, decidió aplicar el principio de cierre por explicación a uno de sus campos de datos más sensibles: el límite de crédito autorizado para cada cliente dentro de su producto de tarjeta de crédito.

El equipo de riesgo y el equipo de tecnología, trabajando en conjunto durante varias semanas, definieron de forma explícita y exhaustiva las únicas tres causas legítimas por las cuales ese campo específico podía cambiar de valor dentro del sistema: primero, una revisión automática anual del perfil crediticio del cliente, ejecutada por un proceso batch programado y auditable; segundo, una solicitud explícita del cliente, formalmente aprobada por un analista de crédito autorizado, con su identificación registrada en el sistema; y tercero, un ajuste automático a la baja por mora, ejecutado según reglas de negocio predefinidas y también completamente auditables.

El equipo de tecnología implementó, sobre esta definición cerrada de causas legítimas, un mecanismo de auditoría automatizado que comparaba, de forma continua, cada cambio efectivamente registrado en el campo de límite de crédito contra el registro de eventos del sistema, verificando que cada cambio correspondiera efectivamente a una de las tres causas autorizadas, con su evidencia correspondiente correctamente registrada. Cualquier cambio que no pudiera vincularse de forma verificable a ninguna de las tres causas generaba, de forma automática e inmediata, una alerta de prioridad alta dirigida al equipo de seguridad de la información.

Dentro de las primeras dos semanas de operación del nuevo mecanismo de auditoría, el sistema detectó un patrón de cambios en el campo de límite de crédito que no correspondía a ninguna de las tres causas legítimas predefinidas: un conjunto de cuentas específicas mostraba incrementos en su límite de crédito sin ningún evento de revisión anual, solicitud de cliente aprobada, o ajuste por mora correspondiente registrado en el sistema. La investigación posterior, iniciada de forma inmediata gracias a la alerta automática, identificó la causa raíz: un error de integración entre dos sistemas internos, introducido durante una actualización técnica reciente, que estaba generando incrementos no autorizados en el límite de crédito de un subconjunto de cuentas bajo ciertas condiciones específicas y poco frecuentes de sincronización entre ambos sistemas. Sin el mecanismo de cierre por explicación implementado, este error —que llevaba, según se determinó posteriormente, varios meses activo antes de su detección— probablemente habría continuado sin ser detectado hasta la siguiente auditoría regulatoria periódica, programada varios meses después, con un potencial de exposición financiera considerablemente mayor al que finalmente se materializó gracias a la detección temprana.

EJERCICIO 11 — LISTA DE CAUSAS LEGÍTIMAS

1. Elige un dato crítico y sensible en tu operación (precio, estatus, saldo, permiso de acceso).
2. Escribe la lista exhaustiva y corta de las únicas causas legítimas por las que ese dato debería cambiar.
3. Pregunta a tu equipo de datos si el sistema actual puede detectar cambios fuera de esa lista. Si no puede, ahí tienes tu próximo proyecto de control.

Aplicalo a tu proyecto

“Un fluent solo cambia si un evento específico lo cambia. Si el evento no ocurrió, el fluent no cambió. Punto.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“La planeación no necesita saber todo lo que no cambia. Necesita una lista corta de lo que sí.”

STRIPS para humanos: listas de qué se agrega y qué se borra

“La planeación no necesita saber todo lo que no cambia. Necesita una lista corta de lo que sí.”

En 1971, Richard Fikes y Nils Nilsson presentaron STRIPS (STanford Research Institute Problem Solver), uno de los sistemas de planeación automática más influyentes en toda la historia de la inteligencia artificial (Fikes & Nilsson, 1971). Su solución al frame problem, aplicada dentro de un contexto de planeación práctica en lugar de razonamiento lógico puro, era radicalmente pragmática: en lugar de razonar de forma explícita sobre absolutamente todo lo que permanece igual después de cada acción, cada acción dentro del sistema se define únicamente mediante tres listas cortas y bien delimitadas. Una lista de precondiciones —lo que debe ser cierto en el mundo para que la acción pueda ejecutarse válidamente—. Una lista de adición (add list) —lo que se vuelve cierto en el mundo, de forma nueva, después de que la acción se ejecuta—. Y una lista de eliminación (delete list) —lo que deja de ser cierto en el mundo después de que la acción se ejecuta—. Todo lo demás, es decir, cualquier propiedad del mundo que no aparezca explícitamente en ninguna de estas dos últimas listas, simplemente permanece igual, sin necesidad de declararlo de forma explícita en ningún lugar del sistema.

Formalmente, si S_0 representa el conjunto de proposiciones que describen el estado actual del mundo, y una acción determinada tiene asociada una lista de adición A y una lista de eliminación D , entonces el estado resultante S_1 se calcula simplemente como $S_1 = (S_0 \cup A) - D$ (Fikes & Nilsson, 1971). Es una de las ideas más simples y, al mismo tiempo, más poderosas de todo este libro, y probablemente la más fácil de instalar de forma inmediata en cualquier equipo, ya desde mañana mismo: para cualquier proceso, evento o transacción relevante de tu negocio, define explícitamente solo tres cosas —qué se requiere que sea

cierto para que ese proceso pueda ejecutarse, qué se agrega al estado del mundo cuando efectivamente ocurre, y qué se elimina o invalida como consecuencia directa—. No definas, en ningún momento, lo que permanece igual; eso se sobreentiende, de forma estructural, por pura exclusión de las dos listas explícitas.

Esta estructura de tres listas es, de hecho, el lenguaje conceptual natural de cualquier buen sistema de eventos o de arquitectura orientada a eventos en la ingeniería de datos moderna: cada evento tiene, típicamente, un esquema explícito de "qué cambió como consecuencia de este evento", y el estado completo del sistema se reconstruye acumulando esos cambios sucesivos sobre una base que persiste por default salvo modificación explícita registrada. Si tu equipo de datos ya trabaja con patrones de arquitectura como event sourcing, o con tablas del tipo slowly changing dimension en un modelo dimensional de datos, ese equipo ya está usando, aunque probablemente sin llamarlo así de forma explícita, una versión moderna y bien evolucionada de la misma estructura conceptual que Fikes y Nilsson propusieron en 1971 para resolver el problema de planeación automática dentro de STRIPS.

La limitación honesta de este enfoque, que los propios Fikes y Nilsson reconocieron desde su formulación original, es que asume que solo una acción ocurre a la vez dentro del sistema y que se conocen de antemano todas las acciones relevantes —exactamente el mismo supuesto restrictivo de ausencia de concurrencia discutido en detalle en el Capítulo 8—. Adicionalmente, el enfoque STRIPS clásico no maneja bien, sin extensiones posteriores considerables, las acciones condicionales, cuyos efectos dependen del estado específico en el que se ejecutan (Pednault, 1991; Penberthy, 1993). Es, por estas razones, una herramienta extraordinariamente poderosa para diseñar procesos internos controlados dentro de tu propia organización, donde efectivamente puedes garantizar razonablemente secuencialidad y conocimiento completo de las acciones relevantes. Es, en cambio, considerablemente más frágil si se aplica de forma directa para intentar modelar el comportamiento del mercado externo, donde la concurrencia, como ya discutimos, es la norma estructural y no la excepción ocasional.

Rediseñar el evento de "venta completada" con STRIPS

Una plataforma de comercio electrónico de mediana escala enfrentaba un problema recurrente de inconsistencia de datos entre sus sistemas de inventario, facturación y programa de lealtad: en un porcentaje no despreciable de transacciones, alguno de estos tres sistemas quedaba desincronizado respecto a los otros dos después de completarse una venta, generando discrepancias que requerían reconciliación manual periódica por parte del equipo de operaciones.

El diagnóstico del problema, realizado por un arquitecto de datos recién incorporado al equipo, reveló que el evento central de "venta completada" dentro del sistema no tenía una definición formal y explícita de qué exactamente debía cambiar como consecuencia de ese evento. Cada uno de los tres sistemas afectados —inventario, facturación, lealtad— había sido desarrollado en momentos distintos, por equipos distintos, y cada uno había implementado su propia interpretación, ligeramente diferente entre sí, de qué debía ocurrir cuando se completaba una venta, generando pequeñas inconsistencias que se acumulaban con el tiempo.

El arquitecto propuso, aplicando explícitamente la estructura de STRIPS descrita en este capítulo, una redefinición formal del evento "venta completada" mediante las tres listas correspondientes. La lista de precondiciones especificaba, de forma explícita: un carrito de compra con al menos un producto válido, un método de pago verificado y autorizado, y disponibilidad de inventario confirmada al momento de la confirmación de pago. La lista de adición especificaba, de forma igualmente explícita: un registro de pedido creado en el sistema de facturación, una reserva de inventario creada en el sistema correspondiente, y puntos de lealtad acreditados en la cuenta del cliente según la regla vigente. La lista de eliminación especificaba: el producto retirado del carrito de compra activo del cliente, y la cantidad correspondiente retirada del inventario disponible para nuevas ventas.

La contribución más valiosa de este ejercicio de formalización no fue, sorprendentemente, ninguno de los elementos incluidos explícitamente en las tres listas —esos, en su mayoría, ya estaban implementados correctamente de forma independiente en cada sistema—. Fue la claridad resultante sobre todo lo que, por construcción explícita del modelo, NO debía verse afectado por el evento y por lo tanto podía asumirse persistente por default: el historial de compras previas del cliente, sus preferencias de comunicación registradas, su dirección de envío guardada, su historial de interacciones con servicio al cliente. Antes de la formalización, distintos desarrolladores, sin coordinación explícita entre sí, habían añadido de forma ocasional lógica adicional "por si acaso" dentro del manejo del evento de venta completada, tocando de forma innecesaria algunos de estos campos que, en realidad, no tenían ninguna relación causal legítima con el evento de venta. Al formalizar explícitamente las tres listas y eliminar toda la lógica adicional no justificada por ninguna de ellas, el equipo redujo tanto la complejidad del código responsable de manejar el evento como, de forma medible en los meses siguientes, la tasa de incidentes de desincronización entre los tres sistemas, que cayó a niveles cercanos a cero.

EJERCICIO 12 — MODELA UN PROCESO CON STRIPS

1. Elige un proceso interno recurrente (aprobación de gasto, alta de proveedor, cierre de ticket).
2. Define sus precondiciones, su lista de adición y su lista de eliminación.
3. Revisa si el proceso actual, tal como está documentado, respeta esa disciplina o mezcla información innecesaria.

Aplicalo a tu proyecto

“La planeación no necesita saber todo lo que no cambia. Necesita una lista corta de lo que sí.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“Entre dos historias igual de posibles, prefiere la que no necesita un milagro.”

Motivated Action Theory: prefiere el mundo con menos sorpresas

“Entre dos historias igual de posibles, prefiere la que no necesita un milagro.”

Volvamos al arma cargada del Capítulo 9, porque su resolución formal es uno de los aportes más elegantes de todo el desarrollo posterior del frame problem. Recordemos el dilema exacto: la lógica pura de McCarthy, tal como estaba formulada a mediados de los años ochenta, permitía tanto la historia esperada —el arma permanece cargada durante toda la espera, y la persona muere al ser disparada— como la historia inesperada y en apariencia absurda —el arma se descarga sola, sin ninguna causa identificada, durante el periodo de espera—. Motivated Action Theory (MAT), desarrollada por Lynn Andrea Stein y Leora Morgenstern (Stein & Morgenstern, 1994; Morgenstern & Stein, 1988), propuso un criterio formal elegante para descartar sistemáticamente la segunda historia sin necesidad de prohibirla explícitamente mediante una regla ad hoc: preferir siempre, entre los modelos formalmente posibles y consistentes con los hechos, aquellos donde el menor número de acciones ocurre sin una razón identificable —sin motivación— para haber ocurrido.

En la historia esperada, no hace falta ninguna acción adicional sin motivo explícito: el arma simplemente permanece cargada porque, sencillamente, nada la descargó de forma activa. En la historia inesperada, en cambio, hace falta postular una acción adicional "de la nada" —el arma se descarga sin ninguna causa conocida dentro del modelo—. Al preferir, de forma sistemática y como criterio formal explícito, los modelos que requieren el menor número de acciones no motivadas, la teoría descarta automáticamente, sin necesidad de una prohibición ad hoc, la historia absurda del arma que se descarga espontáneamente.

MAT tiene, además, una ventaja estructural significativa sobre las soluciones causales previas que discutiremos brevemente en este mismo capítulo, y que Morgenstern (1996) destaca de forma específica: a diferencia de esas soluciones, MAT no está construida sobre la situation calculus tradicional, sino sobre una teoría de tiempo basada en intervalos, tomada de un trabajo previo de McDermott (1982), lo cual le permite manejar de forma nativa acciones concurrentes, acciones desconocidas y vacíos de información (gaps) en el conocimiento disponible —precisamente las limitaciones estructurales que, como discutimos en el Capítulo 8, hacen que buena parte de las soluciones basadas en la situation calculus clásica resulten frágiles fuera de contextos controlados y de laboratorio.

Este criterio —preferir sistemáticamente la explicación que requiere el menor número de eventos sin causa conocida, entre un conjunto de explicaciones formalmente compatibles con la evidencia disponible— es, en esencia, una versión formalizada y rigurosa de la navaja de Ockham aplicada específicamente al razonamiento sobre el tiempo y el cambio. Y es, además, una herramienta de decisión sorprendentemente directa y aplicable en el contexto de negocio: cuando dos explicaciones compiten razonablemente por dar cuenta de los mismos datos disponibles, en igualdad relativa de evidencia directa disponible para ambas, prefiere sistemáticamente la explicación que requiere menos "eventos misteriosos sin causa conocida". No porque la otra explicación sea formalmente imposible —con frecuencia no lo es—, sino porque, en ausencia de evidencia adicional que decante la balanza, es la apuesta racional y estadísticamente más razonable a la cual asignar mayor probabilidad inicial.

La aplicación práctica más directa e inmediata de este criterio está en el diagnóstico sistemático de anomalías dentro de cualquier operación de datos o negocio: cuando una métrica relevante se comporta de forma inesperada, en lugar de generar múltiples teorías competidoras de complejidad comparable sin ningún criterio explícito para priorizarlas, ordénalas de forma sistemática según cuántos eventos no observados y sin causa conocida requiere cada una para ser cierta. La teoría que

únicamente necesita postular "un evento conocido, específico, que no habíamos registrado o monitoreado correctamente hasta ahora" es, de forma consistente en la experiencia práctica, considerablemente más probable a priori que la teoría que necesita postular, de forma simultánea, tres o más coincidencias independientes sin ninguna evidencia directa que las respalde individualmente.

EJEMPLO EXTENDIDO

Dos teorías compitiendo por la misma caída de conversión

El equipo de producto de un sitio de comercio electrónico observó, en su tablero de métricas diarias, una caída del veinte por ciento en la tasa de conversión del sitio de un día para otro, un cambio considerablemente por encima del ruido estadístico normal observado históricamente en esa métrica y suficientemente severo como para generar preocupación inmediata dentro del equipo directivo del producto.

En la reunión de emergencia convocada para diagnosticar el problema, surgieron rápidamente dos teorías competidoras, presentadas por distintos miembros del equipo con igual nivel de confianza inicial. La primera teoría, propuesta por un ingeniero de frontend, señalaba que un cambio de diseño en el flujo de pago, desplegado precisamente el día anterior a la caída observada, no había sido probado de forma suficientemente rigurosa en dispositivos móviles, que representaban una proporción mayoritaria del tráfico total del sitio, y que probablemente estaba generando fricción o errores no capturados por las pruebas automatizadas existentes. La segunda teoría, propuesta por un analista de marketing, señalaba una combinación de tres factores simultáneos: un posible problema de conectividad regional afectando a una parte significativa de los usuarios, una caída generalizada en la confianza del consumidor relacionada con noticias económicas recientes, y un posible error de tracking en el sistema de analítica que estuviera subreportando conversiones reales sin afectar el negocio real.

Antes de asignar recursos de investigación a ambas teorías por igual —la respuesta reflexiva inicial del equipo, que hubiera dividido el esfuerzo de diagnóstico de forma poco eficiente entre dos líneas de investigación de complejidad muy distinta—, alguien en la reunión aplicó explícitamente el criterio de acción motivada descrito en este capítulo: contar, para cada teoría, cuántos eventos sin causa conocida o sin evidencia directa disponible requería cada una para ser cierta. La primera teoría requería un único evento con evidencia directa disponible de forma inmediata: el registro del despliegue del cambio de diseño, ocurrido exactamente un día antes de la caída observada, un hecho verificable en minutos consultando el sistema de control de versiones y el registro de despliegues. La segunda teoría requería, en cambio, tres eventos independientes, ninguno de los cuales tenía evidencia directa disponible en ese momento dentro de la sala: nadie había confirmado un problema de conectividad regional específico, nadie tenía evidencia de una caída generalizada de confianza del consumidor más allá de una impresión general, y nadie había verificado si efectivamente existía un error de tracking.

Siguiendo el criterio de priorizar la teoría con menos eventos no motivados, el equipo investigó primero, y de forma exclusiva durante la primera hora, la hipótesis del cambio de diseño en móvil. La investigación confirmó rápidamente el problema: el nuevo diseño del flujo de pago presentaba, efectivamente, un error de renderizado en un subconjunto específico y no menor de dispositivos móviles, que impedía completar el pago de forma correcta en ciertas condiciones de pantalla. El error se corrigió y desplegó en menos de tres horas desde el inicio de la investigación, y la tasa de conversión regresó a sus niveles históricos normales inmediatamente después del despliegue de la corrección. La segunda teoría, más elaborada y en apariencia más "completa" al considerar múltiples factores simultáneos, hubiera consumido considerablemente más tiempo de investigación sin ninguna garantía de resultado, precisamente porque, siguiendo la lógica de Motivated Action Theory, requería postular más eventos sin causa conocida de los estrictamente necesarios para explicar la evidencia disponible.

EJERCICIO 13 — ORDENA TUS HIPÓTESIS

1. Ante la próxima anomalía en una métrica clave, escribe todas las hipótesis que el equipo proponga.
2. Para cada una, cuenta cuántos eventos "sin causa conocida todavía" requiere para ser cierta.
3. Investiga primero las hipótesis con menos eventos no explicados, no las más dramáticas.

Aplícalo a tu proyecto

| “Entre dos historias igual de posibles, prefiere la que no necesita un milagro.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““No defines todo lo normal. Define lo anormal, y minimízalo.””

Circunscripción práctica: minimizar lo anormal

"No defines todo lo normal. Define lo anormal, y minimízalo."

La circunscripción (circumscription), propuesta también por John McCarthy (1980), es una técnica formal de razonamiento de sentido común que subyace, de forma directa o indirecta, a buena parte de las ideas ya desarrolladas en este libro, y merece un capítulo propio porque su lógica subyacente es considerablemente más general de lo que hemos mostrado hasta este punto. La idea central, formulada de manera precisa: en lugar de intentar definir de forma exhaustiva y positiva qué constituye "comportamiento normal" dentro de un dominio determinado, se define un predicado explícito de "anormalidad" (abnormality, típicamente abreviado ab en la notación formal), y se le pide al sistema de razonamiento que asuma, como principio general, la menor cantidad posible de casos considerados anormales, compatible con la evidencia efectivamente disponible.

Aplicado al ejemplo canónico de la literatura sobre aves: en lugar de intentar enumerar exhaustivamente todas las especies de aves que efectivamente vuelan —una lista larga y, en cualquier caso, incompleta—, se establece un principio general de la forma "las aves vuelan, a menos que sean anormales respecto a la propiedad de volar", y posteriormente se minimiza, de forma sistemática, cuántas aves específicas se consideran anormales respecto a esa propiedad particular. Un pingüino, del que se tiene evidencia explícita y directa de que no vuela, se marca formalmente como anormal respecto a esa propiedad. Cualquier ave de la que no se tenga evidencia específica en contrario se asume, por el propio proceso de minimización, como normal respecto al vuelo —y, por lo tanto, se concluye que vuela, salvo que aparezca evidencia específica que indique lo contrario.

El poder práctico de esta técnica para el diseño de sistemas de negocio

está en que invierte por completo la carga de trabajo de documentación requerida: en lugar de que un equipo tenga que documentar de forma exhaustiva "cómo se comporta normalmente cada segmento particular de clientes, producto o proceso", el equipo documenta únicamente, en una frase compacta, la regla general que gobierna el comportamiento esperado por default, y construye, de forma separada y explícita, una lista corta y activamente mantenida de las excepciones conocidas y evidenciadas a esa regla general. Esta inversión reduce de forma drástica el esfuerzo de mantenimiento continuo de reglas de segmentación o clasificación y, quizás más importante todavía, hace explícito y fácilmente auditable, en todo momento, exactamente cuáles segmentos o casos particulares requieren atención especial y diferenciada del resto.

La disciplina operativa concreta derivada de este capítulo: cuando defines un comportamiento esperado para una población relevante de tu negocio —clientes, productos, tiendas, proveedores, transacciones—, evita documentar de forma exhaustiva el comportamiento "normal" caso por caso, subgrupo por subgrupo. Documenta, en cambio, la regla general en una sola frase compacta y clara, y construye, a partir de ahí, una lista explícita, deliberadamente corta y activamente auditada de excepciones documentadas. Minimiza esa lista de excepciones de forma activa y continua a lo largo del tiempo: cada excepción que decides agregar a la lista debe estar respaldada por evidencia específica y verificable, no por una precaución genérica o por el reflejo defensivo de "por si acaso ocurre algo distinto que todavía no hemos observado".

EJEMPLO EXTENDIDO

De cuarenta segmentos a seis excepciones documentadas

El equipo de marketing de una cadena de gimnasios había desarrollado, a lo largo de varios años y a través de sucesivas iniciativas puntuales, un esquema de segmentación de clientes que, para el momento en que un

nuevo director de marketing asumió el área, contenía cuarenta segmentos distintos, cada uno con sus propias reglas de comunicación, frecuencia de contacto y oferta comercial asociada, mantenidas en documentos y configuraciones de sistema dispersas y, en varios casos, mutuamente contradictorias entre sí.

El nuevo director, al intentar entender la lógica subyacente del esquema completo antes de proponer cualquier cambio, descubrió que una proporción considerable de esos cuarenta segmentos no representaba, en realidad, un comportamiento genuinamente distinto y diferenciado de los clientes, sino variaciones menores, en muchos casos redundantes entre sí, sobre lo que en el fondo era el mismo comportamiento general de la base de clientes, capturadas en su momento por distintos analistas que, sin comunicación explícita entre ellos, habían creado segmentos ligeramente distintos para necesidades de campaña muy puntuales y de corta duración que ya habían concluido hacía tiempo, pero cuyos segmentos asociados nunca se habían retirado formalmente del sistema.

Aplicando de forma deliberada el principio de circunscripción descrito en este capítulo, el equipo emprendió un ejercicio de tres semanas para redefinir la segmentación completa bajo un enfoque distinto: en lugar de mantener cuarenta segmentos paralelos, se estableció una única regla general de comportamiento esperado por default para la base completa de clientes activos, respaldada por el análisis agregado del comportamiento histórico observado, y se identificaron, mediante un proceso riguroso de validación estadística contra la evidencia real disponible, únicamente seis grupos de clientes cuyo comportamiento se apartaba, de forma estadísticamente significativa y consistentemente documentada, del comportamiento general esperado por default —por ejemplo, clientes en periodo de prueba gratuita, clientes corporativos con condiciones contractuales especiales, y clientes en proceso activo de cancelación—.

El resultado de este ejercicio de simplificación fue doble. Por un lado, una reducción drástica en el esfuerzo continuo de mantenimiento del esquema de segmentación, que pasó de requerir la atención dispersa de

varios analistas a lo largo del tiempo, a ser gestionado de forma centralizada y con claridad por una sola persona responsable. Por otro lado, y quizás más valioso todavía, una mejora medible en la efectividad de las campañas de comunicación resultantes: al eliminar segmentos redundantes y contradictorios entre sí, se eliminó también un problema previo de clientes que, por pertenecer simultáneamente a múltiples segmentos con reglas de comunicación distintas y no coordinadas, recibían mensajes de marketing contradictorios o excesivamente frecuentes, generando quejas y solicitudes de baja de comunicaciones que, tras la simplificación, se redujeron de forma sustancial en los meses siguientes al cambio.

Aplícalo a tu proyecto

| “No defines todo lo normal. Define lo anormal, y minimízalo.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“La tendencia debería ser modificar e integrar. En cambio, la costumbre es perforar el trabajo ajeno y proponer algo totalmente distinto.”

Integrar en vez de reinventar: construir sobre lo que ya funciona

"La tendencia debería ser modificar e integrar. En cambio, la costumbre es perforar el trabajo ajeno y proponer algo totalmente distinto."

Uno de los patrones más frustrantes documentados en la historia del frame problem —y, al mismo tiempo, uno de los más comunes y reconocibles en la vida organizacional cotidiana— es el ciclo recurrente de "encontrar el hueco específico en la teoría del otro, y proponer una teoría completamente nueva desde cero" en lugar de "reparar el hueco específico identificado dentro de la teoría ya existente y previamente validada en muchos otros aspectos". Cada una de las soluciones formales al frame problem que hemos revisado en esta tercera parte del libro —el principio de persistencia por default, el cierre por explicación, STRIPS, Motivated Action Theory, la circunscripción— resuelve de forma efectiva una parte específica del problema general, y falla, o simplemente no aborda, otra parte igualmente relevante. Ninguna de ellas, tomada de forma aislada, constituye una solución completa. Pero combinadas de forma deliberada y con criterio, según Sandewall (1994) y el propio análisis de Morgenstern (1996), se complementan de forma considerablemente más robusta que cualquiera de ellas por separado.

Este es, probablemente, el mensaje metodológico individual más importante de todo el libro, y el que cierra apropiadamente esta tercera parte dedicada al Toolkit formal: no busques el framework perfecto y universal que resuelva de una sola vez, y de forma definitiva, todo tu problema particular de framing. No existe, ni en la lógica formal académica ni en el contexto práctico de negocio —y la propia historia de más de dos décadas de investigación intensa que hemos recorrido en los capítulos anteriores es la evidencia más contundente disponible de esta afirmación—. Busca, en cambio, combinar de forma deliberada y con

criterio explícito varias de las herramientas de este Toolkit, según la naturaleza específica y las particularidades del problema concreto que tienes frente a ti en cada situación.

Usa el principio de persistencia por default del Capítulo 13 para reducir la carga documental asociada a todo lo genuinamente estable dentro de tu sistema. Usa el cierre por explicación del Capítulo 14 específicamente para los datos críticos y sensibles donde necesitas capacidad de auditoría explícita sobre las causas legítimas de cambio. Usa la estructura de tres listas de STRIPS del Capítulo 15 para modelar procesos internos secuenciales y razonablemente controlados dentro de tu propia organización. Usa el criterio de acción motivada de Motivated Action Theory del Capítulo 16 para priorizar de forma sistemática entre hipótesis competidoras cuando enfrentes anomalías inesperadas. Usa la circunscripción del Capítulo 17 para documentar poblaciones grandes y heterogéneas de tu negocio mediante excepciones acotadas, en lugar de mediante reglas exhaustivas imposibles de mantener.

Ninguna de estas cinco herramientas, tomada de forma aislada, constituye por sí sola un framework universal y suficiente de decisión para cualquier situación que puedas enfrentar. Combinadas con criterio, según la naturaleza específica del problema concreto que tienes enfrente, sí constituyen el Toolkit completo que este libro te prometió desde su capítulo introductorio. La cuarta parte del libro, que comienza a continuación, te muestra con detalle operativo exactamente cómo combinar estas cinco herramientas frente a problemas reales y concretos de datos, inteligencia artificial y negocio.

EJERCICIO 14 — TU COMBINACIÓN DE HERRAMIENTAS

1. Elige un problema real que tu equipo enfrenta hoy.
2. Para cada una de las cinco herramientas del Toolkit (persistencia, cierre por explicación, STRIPS, acción motivada, circunscripción), decide si aplica, parcialmente o no aplica.
3. Diseña, en una página, cómo combinarías las herramientas que sí aplican.

Aplicalo a tu proyecto

“La tendencia debería ser modificar e integrar. En cambio, la costumbre es perforar el trabajo ajeno y proponer algo totalmente distinto.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

Antes de continuar

¿Cuál de las cinco herramientas del Toolkit aplicarías primero, y en qué proyecto concreto?

TU RESPUESTA

UNA ACCIÓN CONCRETA PARA ESTA SEMANA

Datos, IA y negocio

Aplicación práctica

“Todo producto de datos es, antes que nada, un ejercicio de framing.”

“El error más caro en un producto de datos no está en el código. Está en la primera junta, cuando nadie preguntó qué se iba a ignorar.”

Framear un producto de datos antes de construirlo

“El error más caro en un producto de datos no está en el código. Está en la primera junta, cuando nadie preguntó qué se iba a ignorar.”

La mayoría de los productos de datos que fracasan, o que simplemente decepcionan las expectativas con las que fueron aprobados, no fracasan por un problema técnico identificable en retrospectiva. Fracasan porque el equipo empezó a construir antes de responder, de forma explícita y documentada, tres preguntas de framing que, sobre el papel, parecen casi obvias: ¿qué situación, en el sentido preciso definido en el Capítulo 4, estamos fotografiando exactamente?; ¿qué fluents nos importa capturar con precisión y cuáles decidimos, de forma consciente, dejar fuera del alcance del proyecto?; y ¿qué acciones —del negocio, del usuario, del mercado— pueden invalidar, en algún momento futuro, lo que hoy estamos asumiendo como estable? Sin respuestas explícitas y documentadas a estas tres preguntas, el equipo técnico empieza, de forma casi inevitable, a tomar esas mismas decisiones de framing de manera implícita y fragmentada, campo por campo, línea de código por línea de código, sin que nadie lo note de forma consciente hasta que el producto ya está en producción y el costo de corregir el framing original se ha multiplicado considerablemente.

Aquí es exactamente donde el Toolkit desarrollado en la Parte III se vuelve directamente operativo y aplicable en el trabajo cotidiano de un equipo de datos. Antes de escribir la primera línea de código de cualquier producto de datos de cierta relevancia, un equipo bien entrenado en las prácticas de este libro produce un documento breve — no un documento de requerimientos de cuarenta páginas, sino, idealmente, una sola página bien estructurada— con cuatro secciones claramente diferenciadas: la lista de fluents volátiles que el producto debe capturar con precisión y frecuencia adecuada; la lista de fluents

estables que el producto puede razonablemente asumir por default, siguiendo el principio desarrollado en el Capítulo 13; la lista de causas legítimas de cambio para cada dato crítico del producto, siguiendo el principio de cierre por explicación del Capítulo 14; y el criterio explícito de anormalidad que debe activar, de forma automática o semiautomática, una revisión completa del modelo o del producto cuando se detecte.

Esta disciplina de framing previo previene sistemáticamente el error más común y, en mi experiencia trabajando con equipos de analítica y ciencia de datos en organizaciones de escala considerable, más costoso en el ámbito de la analítica aplicada: construir un modelo que funciona de forma impecable sobre los datos históricos disponibles en el momento del desarrollo, precisamente porque esos datos históricos reflejan un framing implícito que, en algún punto posterior no anticipado, dejó de ser válido sin que el modelo tuviera ningún mecanismo para saberlo. Un modelo de propensión de compra entrenado sobre datos generados por un canal de venta que, posteriormente, cambió de forma sustancial su experiencia de usuario, no está fallando por una razón técnica identificable en el código o en el algoritmo de entrenamiento; está operando, con toda precisión matemática interna, bajo un framing que el mundo real ya abandonó silenciosamente.

La recomendación operativa concreta para tu próximo proyecto de datos: antes de la primera reunión formal de kickoff técnico con el equipo de desarrollo, agenda una sesión de framing dedicada, de no más de noventa minutos de duración, usando la plantilla de las cuatro secciones descritas en este capítulo —y que encontrarás, en formato de ficha de trabajo lista para usar, en el Apéndice A al final de este libro—. El costo de esa sesión, medido en tiempo del equipo, es de una tarde. El ahorro correspondiente, medido en retrabajos evitados y en credibilidad organizacional preservada, se cuenta consistentemente en meses de trabajo posterior no necesario.

EJEMPLO EXTENDIDO

El motor de recomendación que ignoró un cambio de catálogo

Una plataforma de comercio electrónico especializada en productos de moda desarrolló, a lo largo de aproximadamente un año de trabajo iterativo, un motor de recomendación de productos que había logrado, según las métricas internas de evaluación del equipo de ciencia de datos, un desempeño consistentemente superior al de la competencia directa dentro de su categoría específica, medido en tasa de clic y en tasa de conversión sobre las recomendaciones presentadas al usuario.

El motor de recomendación había sido entrenado, y posteriormente reentrenado de forma periódica, sobre una estructura de categorías de producto que había permanecido estable durante los dos años previos de operación del catálogo. Esa estabilidad estructural, nunca cuestionada de forma explícita por el equipo de ciencia de datos durante el desarrollo del modelo, se había convertido, con el tiempo, en un supuesto implícito y no documentado del sistema: el modelo asumía, en su propia arquitectura interna de relaciones entre categorías, que la taxonomía de productos permanecería estructuralmente equivalente indefinidamente hacia adelante.

El área de marca y experiencia de cliente, de forma independiente y sin coordinación previa con el equipo de ciencia de datos, decidió emprender una reestructuración completa del catálogo de categorías como parte de un relanzamiento integral de la marca, con el objetivo explícito de mejorar la navegación del sitio y alinear la taxonomía de producto con nuevas tendencias de búsqueda observadas en el comportamiento reciente de los consumidores. La reestructuración, desde la perspectiva del área que la propuso e implementó, era un proyecto de experiencia de usuario completamente independiente del motor de recomendación, sin ninguna relación aparente con el trabajo del equipo de datos.

Tras el relanzamiento del catálogo reestructurado, el motor de recomendación continuó operando, sin ninguna interrupción visible en

su funcionamiento técnico, pero recomendando activamente productos usando relaciones de categoría que ya no correspondían a la nueva estructura vigente del catálogo, generando recomendaciones que, para el usuario final, resultaban crecientemente incoherentes y, en algunos casos, directamente contradictorias con la nueva navegación del sitio. La tasa de clic sobre las recomendaciones cayó de forma progresiva a lo largo de varias semanas, sin que ninguna alerta automática se disparara, porque el sistema de monitoreo existente solo vigilaba métricas de desempeño agregadas, no la validez estructural de los supuestos subyacentes del modelo. Cuando finalmente se identificó la causa raíz del deterioro, varias semanas después del relanzamiento del catálogo, el reentrenamiento completo del modelo bajo la nueva estructura tomó considerablemente más tiempo del que hubiera tomado incorporar, desde el diseño original del proyecto de reestructuración del catálogo, una simple notificación al equipo de ciencia de datos sobre el cambio planeado. La causa raíz, en última instancia, no fue ningún error técnico identificable en ninguno de los dos proyectos por separado; fue la ausencia de un documento de framing que hubiera identificado explícitamente la estructura de categorías del catálogo como un fluent condicional de alto riesgo, sujeto a decisiones de otras áreas de la organización, que merecía monitoreo activo y coordinación explícita entre equipos.

EJERCICIO 15 — FRAMING CANVAS DE UNA PÁGINA

1. Antes de tu próximo proyecto de datos, completa: fluents volátiles a capturar, fluents estables asumidos, causas legítimas de cambio del dato crítico, criterio de anormalidad para disparar revisión.
2. Comparte el documento con el equipo técnico y con el dueño de negocio antes del kickoff.
3. Guarda el documento junto al modelo; revísalo cada vez que el modelo se reentrene.

Aplicalo a tu proyecto

“El error más caro en un producto de datos no está en el código. Está en la primera junta, cuando nadie preguntó qué se iba a ignorar.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Un modelo de lenguaje no sabe qué olvidar. Tú tienes que decírselo.””

IA generativa: la ventana de contexto es tu situation calculus

“Un modelo de lenguaje no sabe qué olvidar. Tú tienes que decírselo.”

Cuando trabajas con un modelo de lenguaje de gran escala —para un asistente interno, un chatbot de atención al cliente, o un copiloto de análisis de datos—, estás, sin llamarlo explícitamente así, diseñando en tiempo real una instancia práctica de la situation calculus que introducimos en el Capítulo 3. Cada mensaje intercambiado con el modelo constituye, en este sentido preciso, una acción. El historial acumulado de la conversación constituye la situación acumulada resultante de esa secuencia de acciones. Y el contexto que decides inyectar de forma deliberada al modelo —documentos de referencia, memoria persistente entre sesiones, instrucciones de sistema— constituye, exactamente, el conjunto completo de fluents que el modelo tiene la capacidad efectiva de considerar en su razonamiento. Todo aquello que no está presente en ese contexto proporcionado, para el modelo, sencillamente no existe como información disponible, sin importar cuán "obvio" pueda parecerte a ti, como diseñador humano del sistema, que debería conocerlo.

Aquí el frame problem reaparece con una textura técnica distinta, propia de la arquitectura estadística de los modelos de lenguaje contemporáneos, pero estructuralmente reconocible: los modelos de lenguaje no cuentan, por diseño arquitectónico nativo, con ningún mecanismo confiable y verificable de persistencia selectiva. No saben, de forma estructuralmente garantizada, qué información proporcionada en el tercer turno de una conversación sigue siendo relevante y válida en el turno veinte, y qué información específica quedó obsoleta porque el usuario la corrigió explícitamente en el turno doce. Sin un diseño explícito de gestión de estado y contexto, construido de forma deliberada por quien diseña el sistema, el modelo tiende a tratar toda la

información contenida en su ventana de contexto como igualmente vigente y relevante —una violación directa, en la práctica, del principio de cierre por explicación desarrollado en el Capítulo 14: idealmente, un dato solo debería cambiar de valor efectivo cuando existe una causa explícita de cambio, y el modelo, por sí solo y sin ayuda de una capa de gestión de estado externa, no siempre tiene la capacidad de distinguir de forma confiable entre una corrección explícita del usuario y una simple instrucción adicional acumulativa.

La aplicación práctica del Toolkit desarrollado en este libro resulta, en este contexto, particularmente concreta y accionable: diseña tus sistemas de inteligencia artificial generativa incorporando una capa explícita y separada de gestión de estado, distinta del modelo de lenguaje mismo, que aplique de forma sistemática el principio de cierre por explicación a la conversación completa. Cuando el usuario corrige un dato previamente proporcionado, ese evento específico —la corrección— debe invalidar de forma explícita y verificable el dato anterior dentro del contexto que efectivamente se reenvía al modelo en la siguiente interacción, en lugar de confiar, sin ninguna garantía estructural, en que el modelo "se dé cuenta" por sí mismo de la corrección entre potencialmente cientos o miles de tokens de historial acumulado.

Esta misma lógica de framing aplica, con igual relevancia, a los sistemas de recuperación aumentada por generación (retrieval-augmented generation, o RAG): cada documento que el sistema recupera de su base de conocimiento constituye, en el sentido preciso del Capítulo 4, una fotografía de una situación pasada, capturada en el momento en que ese documento fue redactado o incorporado a la base. Si tu base de conocimiento no cuenta con un mecanismo explícito de cierre por explicación —es decir, una forma explícita y sistemática de marcar qué documentos previos quedaron obsoletos, y bajo qué causa específica quedaron obsoletos—, tu sistema de inteligencia artificial generativa recuperará, con total confianza aparente en su respuesta, información que ya no es vigente ni correcta, y la presentará al usuario con exactamente el mismo tono de seguridad y autoridad que emplearía para presentar información plenamente vigente y verificada.

El asistente interno que citó, con total confianza, una política ya derogada

Una empresa de tamaño considerable implementó un asistente interno basado en un modelo de lenguaje con recuperación aumentada, diseñado para responder preguntas frecuentes de empleados sobre políticas de recursos humanos, beneficios y procedimientos administrativos internos, con el objetivo de reducir la carga de tickets rutinarios sobre el equipo humano de recursos humanos.

El sistema, en su diseño original, incorporaba a su base de conocimiento la totalidad de los documentos de política vigentes en el momento de su implementación inicial, correctamente indexados y disponibles para recuperación. El equipo de tecnología que implementó el sistema, sin embargo, no diseñó, en esa etapa inicial del proyecto, ningún mecanismo formal para gestionar la actualización o el retiro de documentos cuando las políticas subyacentes cambiaran con el tiempo, un evento que, en cualquier organización de tamaño considerable, ocurre con razonable regularidad.

Cuatro meses después de la implementación inicial del sistema, el área de recursos humanos actualizó formalmente su política de días de vacaciones acumulables, ampliando de forma significativa el número máximo de días que un empleado podía acumular de un año a otro. El nuevo documento de política se agregó a la base de conocimiento del sistema siguiendo el proceso operativo estándar de actualización de contenido. El documento anterior, correspondiente a la política ya derogada, sin embargo, nunca se marcó explícitamente como obsoleto ni se retiró de la base de conocimiento; simplemente permaneció indexado junto al nuevo documento, sin ninguna relación explícita de sustitución entre ambos registrada en el sistema.

Cuando empleados consultaban al asistente sobre la política vigente de acumulación de vacaciones, el sistema de recuperación, sin ningún

criterio explícito de vigencia temporal incorporado en su lógica de búsqueda, recuperaba en ocasiones el documento correcto y actualizado, y en otras ocasiones —dependiendo de detalles de la formulación específica de la pregunta del usuario y de las particularidades del algoritmo de similitud semántica empleado internamente— recuperaba el documento derogado, generando respuestas incorrectas presentadas con exactamente el mismo tono de autoridad y confianza que las respuestas correctas. El problema, descubierto únicamente cuando un empleado reportó al equipo de recursos humanos una discrepancia entre lo que el asistente le había informado y lo que su gerente le confirmó verbalmente, había estado activo durante varias semanas antes de su detección, generando un número indeterminado de decisiones de planeación personal de empleados basadas en información incorrecta. La remediación requirió no solo corregir el caso específico detectado, sino implementar, de forma retroactiva sobre la base de conocimiento completa ya existente, exactamente el mecanismo de cierre por explicación que este capítulo recomienda incorporar desde el diseño original: cada documento nuevo que reemplaza formalmente a otro debe invalidar de forma explícita y verificable al documento anterior dentro del propio sistema, no simplemente coexistir con él de forma ambigua.

EJERCICIO 16 — AUDITORÍA DE CONTEXTO DE TU ASISTENTE DE IA

1. Revisa la base de conocimiento de tu sistema de IA generativa o RAG.
2. Identifica si existe un mecanismo explícito para marcar documentos como obsoletos y su causa.
3. Si no existe, diseña uno: cada documento nuevo que reemplace a otro debe invalidar explícitamente al anterior.

Aplícalo a tu proyecto

| “Un modelo de lenguaje no sabe qué olvidar. Tú tienes que decírselo.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“El piso de venta es el entorno concurrente más denso que vas a modelar. Ahí, la concurrencia nunca es la excepción.”

Retail y operaciones: qué persiste, qué cambia, quién lo causa

“El piso de venta es el entorno concurrente más denso que vas a modelar. Ahí, la concurrencia nunca es la excepción.”

El retail es, posiblemente, el dominio de negocio donde el *frame problem* se manifiesta con mayor intensidad práctica, precisamente porque constituye un entorno de altísima concurrencia estructural: miles de acciones distintas —ventas, devoluciones, ajustes de inventario, cambios de precio, promociones lanzadas por la competencia— ocurren de forma simultánea a lo largo de cualquier jornada operativa normal, y una proporción considerable de esas acciones ocurre sin ningún registro directo o inmediato dentro de los sistemas centrales de la organización. Cualquier modelo de datos o sistema de decisión que asuma, de forma implícita o explícita, acciones únicas y completamente conocidas —como discutimos en detalle en el Capítulo 8— está estructuralmente condenado a fallar en algún punto del ciclo comercial, particularmente durante periodos de alta actividad como temporadas de venta especiales.

La aplicación disciplinada del Toolkit desarrollado en la Parte III, aplicada específicamente al contexto de retail, comienza por separar de forma explícita, para cada dato operativo clave de la operación, las tres listas descritas originalmente en el Capítulo 4: qué es volátil por naturaleza estructural del negocio —inventario disponible, precio vigente en tienda, disponibilidad inmediata de un producto—; qué es condicional —el estatus de una promoción comercial, activo únicamente durante su ventana de vigencia formalmente autorizada—; y qué es genuinamente estable dentro del horizonte razonable de decisión relevante —la ubicación geográfica física de una tienda, salvo un evento explícito de relocalización—. Este ejercicio de clasificación, en apariencia simple y directo, es precisamente el que la mayoría de los proyectos de analítica de retail que he observado tienden a saltarse en la

práctica, apresurados por presión de tiempo, y es la causa raíz identificable de buena parte de los "dashboards que mienten sin mentir" que ya discutimos en detalle en el Capítulo 1.

El principio de cierre por explicación desarrollado en el Capítulo 14 tiene una aplicación particularmente valiosa en el contexto específico del control de inventario en retail: define de forma explícita las únicas causas legítimas por las cuales el inventario disponible reportado puede cambiar de valor —una venta registrada, una devolución procesada, un ajuste derivado de un conteo físico periódico, un traspaso formal entre tiendas de la misma cadena, o una merma registrada según el proceso operativo correspondiente—. Cualquier discrepancia de inventario detectada que no corresponda a ninguna de estas causas previamente enumeradas constituye, por la propia construcción lógica del sistema, una señal directa de fuga, error de sistema, o, en casos más graves, fraude, y debería activar de forma automática una alerta prioritaria, en lugar de depender, como ocurre todavía en un número considerable de operaciones de retail, de una auditoría manual trimestral que puede tardar meses en detectar el problema.

Finalmente, el criterio de acción motivada derivado de Motivated Action Theory, desarrollado en el Capítulo 16, resulta especialmente útil para el diagnóstico ágil de quiebres de inventario inesperados: ante una caída inesperada de ventas observada en una categoría específica, prioriza sistemáticamente la investigación de la hipótesis que requiere menos eventos sin causa conocida —típicamente, un quiebre de inventario no registrado correctamente en el sistema central— antes de invertir tiempo considerable investigando hipótesis más complejas relacionadas con un cambio genuino en las preferencias de consumo del cliente, que típicamente requerirían, para ser ciertas, múltiples eventos simultáneos y correlacionados sin evidencia directa disponible que los respalde de forma individual.

EJEMPLO EXTENDIDO

Las tres listas aplicadas a una cadena de conveniencia nacional

Una cadena de tiendas de conveniencia con presencia en más de mil puntos de venta a nivel nacional emprendió, como parte de una iniciativa de modernización de su plataforma de datos, un ejercicio formal de reclasificación de sus principales fuentes de datos operativos siguiendo explícitamente la metodología de las tres listas desarrollada en el Capítulo 4 de este libro: fluentes volátiles, condicionales y estables.

En la lista de fluentes volátiles, el equipo clasificó correctamente el inventario disponible por SKU y por tienda individual, actualizado en tiempo real con cada transacción de venta procesada en el punto de venta. Esta clasificación implicaba, de forma directa, que ningún proceso de decisión operativa debía asumir, bajo ninguna circunstancia, que un valor de inventario capturado hace más de unos minutos seguía siendo plenamente confiable sin una verificación adicional en contextos de alta rotación.

En la lista de fluentes condicionales, el equipo clasificó el precio de venta al público de cada producto, correctamente identificado como una propiedad que cambia, pero únicamente durante ventanas de tiempo específicas y formalmente autorizadas por el proceso de aprobación de promociones comerciales de la cadena, no de forma continua ni arbitraria. Esta clasificación permitió al equipo de tecnología diseñar un sistema de auditoría de precios que verificaba, de forma automática, que cada cambio de precio efectivamente registrado en el sistema correspondiera a una promoción formalmente aprobada dentro del calendario comercial vigente.

En la lista de fluentes estables, el equipo clasificó, entre otras propiedades, el layout físico general de categorías dentro del punto de venta, correctamente identificado como una propiedad que solo cambia con eventos infrecuentes y de gran escala, como una remodelación formal de tienda, y que por lo tanto podía asumirse de forma razonable como constante para efectos de la mayoría de los modelos operativos de

la cadena, sin necesidad de verificación continua.

La aplicación disciplinada de esta clasificación, combinada de forma directa con el principio de cierre por explicación aplicado específicamente al flúent volátil de inventario, permitió al equipo de auditoría interna detectar, en un plazo de apenas tres semanas desde la implementación del nuevo mecanismo de control, un patrón sostenido de merma no registrada correctamente en dos tiendas específicas de la cadena, un problema que, según se determinó en la investigación posterior, llevaba varios meses activo sin ser detectado bajo el esquema de auditoría manual previo, precisamente porque ese esquema anterior no contaba con ningún mecanismo automático capaz de distinguir de forma sistemática entre las causas legítimas de cambio de inventario y las discrepancias que no correspondían a ninguna de ellas.

Aplicalo a tu proyecto

“El piso de venta es el entorno concurrente más denso que vas a modelar. Ahí, la concurrencia nunca es la excepción.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“No puedes eliminar la competencia. Puedes diseñar puntos de revisión que la sobrevivan.”

Diseñar decisiones a prueba de concurrencia

"No puedes eliminar la concurrencia. Puedes diseñar puntos de revisión que la sobrevivan."

Si el Capítulo 8 estableció que la concurrencia constituye una condición estructural e ineliminable del mundo real dentro del cual opera cualquier organización, este capítulo ofrece un método concreto y directamente aplicable para diseñar decisiones que no dependan, de forma implícita, de la fantasía conveniente pero falsa de un mundo que permanece quieto mientras se ejecuta un plan. La técnica, que dentro de este libro llamaremos diseño por puntos de revisión motivados, consiste en lo siguiente: en lugar de un plan estrictamente lineal que asume, de forma implícita, validez constante desde el día uno de su aprobación hasta el día final de su ejecución, el plan incorpora, desde su diseño original, puntos de revisión explícitos y calendarizados, cada uno de ellos vinculado de forma directa y específica a un conjunto acotado de flujos que, con razonable probabilidad, podrían haber cambiado como consecuencia de acciones concurrentes fuera del control directo de quien diseñó el plan.

La diferencia sustancial entre este enfoque y una revisión de proyecto genérica —del tipo "nos vemos en la junta mensual regular de seguimiento del proyecto"— es que el punto de revisión motivado, tal como se propone en este capítulo, no revisa el proyecto completo de forma indiscriminada y superficial. Revisa, de forma específica y dirigida, exactamente los supuestos de persistencia que el plan original identificó explícitamente como críticos para su viabilidad: si el plan asumió, como condición relevante de éxito, que el tipo de cambio se mantendría dentro de un rango razonablemente estable, el punto de revisión correspondiente pregunta de forma explícita y verificable por el comportamiento reciente del tipo de cambio, no por "cómo va todo en

general" con el proyecto.

Para construir estos puntos de revisión de forma sistemática, retoma directamente el documento de framing desarrollado siguiendo la metodología del Capítulo 19: cada fluent que se clasificó, en ese documento inicial, como "condicional" o como "estable, pero de riesgo relativamente alto", se convierte, de forma directa, en un ítem específico y explícito dentro de la agenda calendarizada de revisión, acompañado de una pregunta concreta y verificable: ¿sigue siendo válido este supuesto específico a la fecha de esta revisión? Si la respuesta obtenida es negativa, el plan cuenta, idealmente, con una ruta de ajuste predefinida y ya discutida con anticipación, en lugar de tener que improvisarla bajo presión considerable en el momento mismo de la crisis, cuando la calidad de las decisiones tomadas bajo presión de tiempo tiende a deteriorarse de forma sistemática.

Este método transforma la gestión de riesgo organizacional, que en un número considerable de organizaciones sigue funcionando como un ejercicio genérico, ritual y de escaso valor práctico real, en una práctica directamente conectada y trazable a los supuestos específicos y explícitamente declarados de cada decisión particular —exactamente lo que la propia estructura formal del frame problem, revisada a lo largo de todo este libro, nos enseña que resulta necesario: no vigilar absolutamente todo por igual, de forma indiferenciada y con el mismo nivel de esfuerzo, sino vigilar, con precisión deliberada y recursos proporcionalmente asignados, específicamente aquello que se declaró explícitamente que se estaba asumiendo como estable.

EJERCICIO 17 — DISEÑA TUS PUNTOS DE REVISIÓN MOTIVADOS

1. Toma el plan de un proyecto activo con horizonte de más de un trimestre.
2. Lista los tres o cuatro supuestos de persistencia más críticos del plan (los que, si se rompen, cambian la estrategia).
3. Define, para cada uno, un punto de revisión específico en el calendario, con la pregunta exacta que se debe responder, y la ruta de ajuste predefinida si la respuesta es negativa.

Aplicalo a tu proyecto

“No puedes eliminar la concurrencia. Puedes diseñar puntos de revisión que la sobrevivan.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““El proyecto no falló por falta de talento. Falló porque nadie preguntó qué se estaba dando por hecho.””

Casos: cuando ignorar el framing costó caro

“El proyecto no falló por falta de talento. Falló porque nadie preguntó qué se estaba dando por hecho.”

Este capítulo reúne, en formato compuesto y cuidadosamente anonimizado a partir de patrones observados de forma recurrente en distintas industrias y organizaciones, tres casos representativos, cada uno organizado según cuál herramienta específica del Toolkit desarrollado en la Parte III habría prevenido o mitigado de forma significativa el problema si se hubiera aplicado con la debida antelación.

Caso 1 — Falta de persistencia por default explícita. Una aseguradora de tamaño mediano emprendió la migración completa de su sistema central de administración de pólizas hacia una plataforma tecnológica más moderna. Al no declarar de forma explícita, como parte del proceso formal de migración, qué campos específicos debían persistir por default entre el sistema anterior y el sistema nuevo, la organización perdió, de forma silenciosa y sin ninguna alerta inmediata, el historial completo de siniestros de clientes con más de diez años de antigüedad en la relación contractual. El equipo técnico responsable de la migración había ejecutado correctamente, y de forma verificable, cada elemento explícitamente incluido en su checklist formal de migración; el problema real fue que ese checklist nunca incluyó, desde su diseño original, una revisión explícita y deliberada de qué información se asumía que persistiría sin necesidad de un proceso de migración activo específicamente diseñado para ella.

Caso 2 — Falta de cierre por explicación en un dato sensible. Una empresa de tecnología financiera descubrió, aproximadamente ocho meses después de que el problema comenzara a manifestarse, que un porcentaje no despreciable de cuentas de clientes había cambiado de estatus de riesgo crediticio sin ninguna causa formalmente registrada

dentro del sistema de auditoría existente. La causa raíz, identificada finalmente tras una investigación considerable, resultó ser un proceso automatizado de mantenimiento técnico que actualizaba campos relacionados dentro de la base de datos sin pasar por el mecanismo oficial y auditado de gestión de eventos de negocio. Sin una lista explícita y formalmente definida de causas legítimas de cambio para el campo específico de estatus de riesgo, ningún miembro del equipo pudo detectar la anomalía por sí mismo, hasta que un regulador externo la señaló durante una revisión de cumplimiento normativo rutinaria.

Caso 3 — Ignorar de forma explícita la concurrencia estructural del entorno. Una cadena regional de restaurantes lanzó una promoción comercial de alcance nacional con una duración planeada de seis semanas, sin incorporar en su diseño ningún punto de revisión intermedio, asumiendo de forma completamente implícita la estabilidad del costo de un insumo clave durante toda la duración de la campaña. Una disrupción regional en la cadena de suministro de ese insumo específico —una acción concurrente y externa, no anticipada por el equipo de planeación de la campaña— disparó su costo a aproximadamente el doble a mitad del periodo de duración de la promoción. Sin ningún punto de revisión motivado específicamente diseñado para vigilar ese supuesto de costo, la promoción continuó operando con margen negativo durante casi tres semanas adicionales antes de que alguien lo detectara, de forma tardía, en el proceso regular de cierre financiero mensual.

EJERCICIO 18 — DIAGNOSTICA TU PROPIO CASO

1. Piensa en un proyecto que haya fallado o se haya retrasado significativamente en tu organización.
2. Identifica cuál de las herramientas del toolkit —persistencia, cierre por explicación, concurrencia, ramificaciones— habría prevenido o mitigado el problema.
3. Documenta el caso en una página y compártelo con tu equipo como aprendizaje, no como señalamiento.

Aplícalo a tu proyecto

“El proyecto no falló por falta de talento. Falló porque nadie preguntó qué se estaba dando por hecho.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“La mejor señal de un buen framing no es que nunca haya sorpresas. Es que las sorpresas cuestan un día, no un trimestre.”

Casos: cuando framear bien ahorró meses

“La mejor señal de un buen framing no es que nunca haya sorpresas. Es que las sorpresas cuestan un día, no un trimestre.”

Así como el capítulo anterior mostró, con el detalle que merece, el costo real de ignorar el framing explícito, este capítulo muestra el valor operativo concreto de aplicarlo de forma disciplinada, con tres ejemplos igualmente compuestos y representativos de patrones observados de forma recurrente.

Caso 1 — Framing canvas aplicado antes de construir. Un equipo de datos de una empresa de logística aplicó, de forma disciplinada, exactamente la metodología descrita en el Capítulo 19 antes de comenzar la construcción de un modelo de estimación de tiempos de entrega para su red de distribución. La sesión de framing de noventa minutos, realizada con participación de representantes de operaciones y no solo del equipo técnico, identificó que "condiciones climáticas específicas por ruta de distribución" constituía un fluent condicional de alto impacto potencial, completamente ausente en el diseño original que el equipo técnico había propuesto de forma independiente. Incorporar esta variable desde el inicio del desarrollo del modelo requirió, según estimación del propio equipo, aproximadamente dos semanas adicionales de desarrollo técnico; descubrir la omisión después del lanzamiento en producción del modelo, según la misma estimación del equipo, habría requerido un trimestre completo de reentrenamiento y, más costoso todavía, una pérdida considerable de confianza del área de negocio en la validez general del modelo.

Caso 2 — Puntos de revisión motivados aplicados de forma sistemática. Una empresa de bienes de consumo, tras adoptar de forma sistemática la práctica desarrollada en el Capítulo 22, incluyó explícitamente en su plan de lanzamiento trimestral un punto de revisión específico dedicado

a verificar el supuesto de estabilidad del tipo de cambio, con una ruta de ajuste ya predefinida y previamente discutida con el comité correspondiente. Cuando el tipo de cambio se movió de forma significativa a mitad del trimestre —un evento estructuralmente idéntico al que dañó de forma considerable a la empresa descrita en el ejemplo extendido del Capítulo 8—, el equipo activó la ruta de ajuste predefinida en cuestión de pocos días desde la detección de la señal, protegiendo el margen del lanzamiento de forma casi completa.

Caso 3 — Cierre por explicación aplicado a un sistema de control de acceso. Un equipo de tecnología de la información, tras aplicar de forma disciplinada la metodología del Capítulo 14 a su sistema interno de gestión de permisos de acceso, definió de forma explícita y exhaustiva las únicas causas legítimas de cambio de nivel de acceso para cualquier usuario del sistema. Un intento posterior de escalación no autorizada de privilegios de acceso, sin ninguna causa correspondiente registrada dentro de la lista predefinida, fue detectado y bloqueado de forma automática por el sistema en cuestión de minutos, en lugar de descubrirse, como habría ocurrido bajo el esquema previo de la organización, en la siguiente auditoría trimestral de seguridad, con una ventana de exposición considerablemente mayor.

El patrón común identificable en los tres casos descritos no es, en ningún sentido relevante, una sofisticación técnica extraordinaria o inusualmente costosa de implementar. Es, de forma consistente en los tres casos, la disciplina relativamente económica de hacer explícitos, con antelación suficiente, los supuestos de persistencia relevantes y las causas legítimas de cambio de los datos críticos, antes de que el sistema correspondiente entrara efectivamente en operación real.

Aplicalo a tu proyecto

“La mejor señal de un buen framing no es que nunca haya sorpresas. Es que las sorpresas cuestan un día, no un trimestre.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

Antes de continuar

¿Qué caso de tu propia experiencia se parece más a los casos de este bloque?

TU RESPUESTA

UNA ACCIÓN CONCRETA PARA ESTA SEMANA

Tu propio framing

Implementación

“Framear no es un talento. Es una disciplina que se instala en equipo.”

““No necesitas un proceso de tres semanas para framear bien. Necesitas una página y noventa minutos, hechos con disciplina.””

El Framework Canvas: una página para framear cualquier problema

*“No necesitas un proceso de tres semanas para framear bien.
Necesitas una página y noventa minutos, hechos con disciplina.”*

Este capítulo consolida la totalidad del Toolkit desarrollado a lo largo de este libro en una única herramienta operativa e integrada: el Framework Canvas. Se trata de una plantilla de una sola página, diseñada específicamente para completarse durante una sesión de framing dedicada, antes de comenzar cualquier proyecto de suficiente relevancia —un producto de datos nuevo, un cambio de política organizacional, un lanzamiento comercial, una decisión estratégica de cierta magnitud—. Contiene seis secciones claramente diferenciadas, cada una de ellas tomada de forma directa de un capítulo específico ya desarrollado anteriormente en el libro.

La primera sección, Situación, correspondiente al Capítulo 4, pregunta: en una frase concisa, ¿qué fotografía exacta del mundo estamos tomando como punto de partida explícito de este proyecto? La segunda sección, Flujos volátiles, correspondiente a los Capítulos 4 y 19, pregunta qué propiedades relevantes cambian con prácticamente cada acción significativa dentro del alcance del proyecto, y deben por lo tanto capturarse con precisión y frecuencia adecuada. La tercera sección, Flujos estables asumidos, correspondiente al Capítulo 13, pregunta qué se asume por default que persiste sin necesidad de verificación activa continua, y qué condición específica de anormalidad rompería ese supuesto si llegara a ocurrir. La cuarta sección, Causas legítimas de cambio, correspondiente al Capítulo 14, solicita, para el dato más crítico identificado dentro del proyecto, la lista corta y exhaustiva de causas válidas y autorizadas de cambio. La quinta sección, Riesgo de concurrencia, correspondiente a los Capítulos 8 y 22, pregunta qué acciones externas, fuera del control directo del equipo, podrían invalidar

los supuestos declarados durante la ejecución del proyecto, y cuáles son, específicamente, los puntos de revisión motivados diseñados para vigilarlas. La sexta y última sección, Ramificaciones esperadas, correspondiente al Capítulo 11, pregunta qué sistemas o indicadores conectados dentro de la organización podrían verse afectados en cascada por el proyecto, y quién, específicamente, debe validar esos efectos antes de proceder con la implementación.

Es importante ser explícito sobre lo que el Framework Canvas no pretende ser: no reemplaza, en ningún sentido, el trabajo analítico o técnico posterior necesario para desarrollar efectivamente el proyecto; es, de forma deliberada, un paso previo a ese trabajo, no un sustituto de él. Su función específica es exactamente la que este libro ha defendido de forma consistente desde su primer capítulo: hacer explícita, con antelación suficiente y de forma documentada, la decisión invisible de qué se va a considerar activamente dentro del alcance del proyecto, y qué, de forma consciente y deliberada, se va a dejar fuera de ese alcance.

En el Apéndice A, al final de este libro, encontrarás la plantilla completa del Framework Canvas, lista para imprimir directamente, fotocopiar, o adaptar sin dificultad a un documento digital compartido dentro de tu propia organización, junto con una versión adicional específicamente diseñada para facilitar sesiones de trabajo colaborativo en equipo.

EJERCICIO 19 — TU PRIMER FRAMEWORK CANVAS

1. Usa la plantilla del apéndice para tu próximo proyecto.
2. Complétala en equipo, no solo, en una sesión de no más de noventa minutos.
3. Guarda el canvas junto a la documentación del proyecto y revísalo en el primer punto de revisión motivado.

Aplicalo a tu proyecto

“No necesitas un proceso de tres semanas para framear bien. Necesitas una página y noventa minutos, hechos con disciplina.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““Toda buena estrategia empieza con una pregunta bien construida, no con una respuesta apurada.””

Checklist HTDQ 2.0: adaptado a proyectos de datos

“Toda buena estrategia empieza con una pregunta bien construida, no con una respuesta apurada.”

Los frameworks clásicos de estrategia de negocio —como el método que combina Héroe, Tesoro, Dragón y Pregunta para construir una narrativa estratégica persuasiva y comunicable, popularizado por Enders y Chevallier en su trabajo sobre resolución estructurada de problemas de negocio— son extraordinariamente útiles para comunicar de forma clara y convincente una estrategia que ya ha sido previamente definida con rigor. Este capítulo propone una adaptación deliberada de esa estructura narrativa, enriquecida de forma explícita con el vocabulario y los conceptos del frame problem desarrollados a lo largo de este libro, específicamente diseñada para proyectos de datos e inteligencia artificial, donde comunicar bien una estrategia no es suficiente si el framing subyacente de esa estrategia nunca se hizo explícito con el rigor adecuado.

El Héroe, dentro de un proyecto de datos, no es necesariamente una persona individual: es, con mayor frecuencia, el sistema o proceso de negocio concreto que el proyecto busca fortalecer de forma medible —el equivalente conceptual a lo que McCarthy y Hayes hubieran denominado la situación inicial relevante—. El Tesoro es el fluent específico que el proyecto busca mejorar de forma medible y verificable: una tasa de conversión, un tiempo de ciclo operativo, una tasa de error del sistema. El Dragón, en la formulación clásica del framework, representa el obstáculo central de la narrativa; en esta adaptación propuesta, corresponde específicamente al supuesto de persistencia más frágil identificado dentro del proyecto —aquel que, si cambia sin haber sido anticipado con antelación suficiente, pone en riesgo real la totalidad del esfuerzo invertido en el proyecto.

La Pregunta, finalmente, combina de forma deliberada las tres piezas narrativas anteriores con una adición crítica, tomada de forma directa de la metodología central de este libro: no basta con formular "¿cómo logramos que el Héroe alcance el Tesoro a pesar del obstáculo que representa el Dragón?" —la formulación clásica del framework original—, sino que se requiere formular, de forma más precisa y operativamente más útil: "¿cómo lo logramos, sabiendo con precisión qué flujos específicos debemos vigilar de forma activa, y cuáles podemos asumir razonablemente que van a persistir sin necesidad de verificación continua?". Esta adición transforma lo que originalmente era una pregunta estratégica meramente inspiradora en una pregunta operativamente framed con precisión suficiente, lista para trasladarse de forma directa al Framework Canvas desarrollado en el capítulo anterior.

Ejemplo aplicado de la estructura completa: "¿Cómo debería el equipo de retención de clientes [el Héroe] reducir la tasa de cancelación (churn) en quince puntos porcentuales durante los próximos dos trimestres [el Tesoro], dado que la definición operativa de 'cliente activo' cambia con cada nueva campaña de marketing sin previo aviso formal al equipo de datos [el Dragón, siguiendo la lógica desarrollada en el ejemplo extendido del Capítulo 4], sabiendo que debemos vigilar de forma activa cualquier cambio futuro en esa definición operativa, y que podemos asumir con razonable seguridad que el patrón de comportamiento transaccional histórico del cliente persiste con estabilidad suficiente [la Pregunta ya correctamente framed]?"

EJERCICIO 20 — CONSTRUYE TU PREGUNTA HTDQ 2.0

1. Identifica el Héroe y el Tesoro de tu próximo proyecto.
2. Identifica el Dragón como el supuesto de persistencia más frágil, no como un obstáculo genérico.
3. Redacta la Pregunta framed completa, incluyendo qué vigilar y qué asumir con seguridad.

Aplicalo a tu proyecto

“Toda buena estrategia empieza con una pregunta bien construida, no con una respuesta apurada.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

““El framing es un deporte de equipo. Hecho en solitario, hereda los puntos ciegos de una sola persona.””

Cómo facilitar una sesión de framing con tu equipo

“El framing es un deporte de equipo. Hecho en solitario, hereda los puntos ciegos de una sola persona.”

Una sesión de framing mal facilitada se convierte, con facilidad, en una lluvia de ideas dispersa y de bajo valor práctico, o, peor todavía, en un ejercicio dominado casi por completo por la persona de mayor jerarquía presente en la sala, cuyos supuestos particulares terminan imponiéndose sobre el resto del grupo sin ser efectivamente cuestionados o contrastados por el resto del equipo. Este capítulo ofrece una estructura de facilitación de noventa minutos de duración total, diseñada específicamente para producir un Framework Canvas robusto y bien fundamentado, con participación genuina y equitativa de la totalidad del equipo relevante para el proyecto en cuestión.

Minutos 0 a 15, trabajo individual y en silencio. Cada participante completa, por su propia cuenta y sin ninguna discusión previa con el resto del grupo, su propia versión de las secciones dos y tres del Framework Canvas —fluents volátiles y fluents estables asumidos, respectivamente—. Este primer paso resulta crítico para la calidad final del ejercicio: evita de forma efectiva que la primera persona en expresar su opinión en voz alta ancle, de forma involuntaria pero significativa, la percepción y las respuestas subsecuentes del resto de los participantes, un sesgo cognitivo de anclaje bien documentado dentro de la literatura sobre dinámicas de trabajo en grupo.

Minutos 15 a 40, comparación y exploración de divergencias. El facilitador de la sesión recopila la totalidad de las listas individuales generadas en el paso anterior y las proyecta juntas ante el grupo completo, sin atribuir de forma explícita cada respuesta a su autor individual, para evitar dinámicas de jerarquía o de defensa personal de posiciones ya expresadas. El foco de esta segunda etapa no está en

identificar las coincidencias entre los distintos participantes —aunque estas también resultan informativas—, sino específicamente en las divergencias observadas: cuando dos participantes distintos clasificaron exactamente el mismo fluent de forma distinta —uno como estable, otro como volátil, por ejemplo—, ahí se encuentra, con notable frecuencia según la experiencia acumulada aplicando esta metodología, el riesgo de framing más relevante y menos obvio del proyecto completo, siguiendo exactamente la misma lógica estructural del diamante de Nixon organizacional discutido en detalle en el Capítulo 10.

Minutos 40 a 70, construcción conjunta de las secciones restantes. Con las divergencias relevantes ya expuestas de forma explícita y discutidas de forma constructiva por el grupo completo, el equipo completa de forma colaborativa las secciones cuatro, cinco y seis del Framework Canvas —causas legítimas de cambio, riesgo de concurrencia, y ramificaciones esperadas—, contando ahora con considerablemente más contexto compartido del que existía al inicio mismo de la sesión de trabajo. Minutos 70 a 90, asignación explícita de responsables. Cada punto de revisión motivado identificado y cada ramificación esperada documentada dentro del canvas necesita, de forma imprescindible, un nombre de responsable individual claramente asignado antes del cierre formal de la sesión de trabajo; sin un responsable explícitamente nombrado para cada elemento, el canvas resultante corre el riesgo real de convertirse en un documento meramente decorativo, archivado sin consecuencias operativas prácticas.

EJERCICIO 21 — FACILITA TU PRIMERA SESIÓN

1. Agenda noventa minutos con el equipo relevante para tu próximo proyecto importante.
2. Sigue la estructura de cuatro bloques descrita en este capítulo.
3. Al cierre, pide retroalimentación anónima sobre si la sesión se sintió abierta o dominada por una sola voz, y ajusta la facilitación la próxima vez.

Aplicalo a tu proyecto

“El framing es un deporte de equipo. Hecho en solitario, hereda los puntos ciegos de una sola persona.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“*Framear mal no se ve como un error. Se ve como confianza excesiva.*”

Errores comunes al framear (y cómo evitarlos)

"Framear mal no se ve como un error. Se ve como confianza excesiva."

Este capítulo funciona, de forma deliberada, como un catálogo estructurado de antipatrones recurrentes, útil como checklist de revisión sistemática antes de dar por cerrado cualquier Framework Canvas dentro de tu organización. Error uno: confundir la ausencia de evidencia disponible con evidencia efectiva de ausencia de cambio, la falacia específica desarrollada en detalle en el Capítulo 12. La señal de alerta correspondiente a este error: supuestos marcados como "estables" dentro del canvas sin ninguna nota explícita sobre cuándo fue la última vez que efectivamente se verificaron.

Error dos: framear en solitario un proyecto que, por su propia naturaleza, afecta a múltiples áreas de la organización. La señal de alerta correspondiente: un Framework Canvas completado en su totalidad por una sola persona, sin las divergencias productivas que, según discutimos en el capítulo anterior, únicamente emergen de forma confiable durante una sesión de trabajo genuinamente colaborativa en equipo. Error tres: sobre corregir hacia la explosión de reglas descrita en el Capítulo 7, intentando declarar de forma exhaustiva absolutamente todo lo que persiste, en lugar de aplicar de forma disciplinada el principio de persistencia por default desarrollado en el Capítulo 13. La señal de alerta correspondiente: un Framework Canvas que se extiende más allá de dos páginas de longitud, una señal prácticamente segura de que el equipo está enumerando de forma innecesaria elementos obvios, en lugar de concentrar su esfuerzo específicamente en lo genuinamente frágil.

Error cuatro: ignorar de forma implícita la concurrencia estructural del entorno, asumiendo que el proyecto se ejecutará dentro de un mundo

efectivamente detenido, el mismo error fundamental discutido en detalle en el Capítulo 8. La señal de alerta correspondiente: un plan de proyecto que no incluye ningún punto de revisión motivado explícito entre su fecha de inicio y su fecha de cierre formal. Error cinco: no asignar responsables explícitos a los supuestos identificados como críticos dentro del canvas, dejando que "alguien" del equipo monitoree de forma difusa lo que, en la práctica cotidiana, termina no siendo monitoreado por nadie en concreto. La señal de alerta correspondiente: secciones del canvas sin ningún nombre de responsable explícitamente asociado a cada riesgo identificado.

Error seis, posiblemente el más sutil y, por esa misma razón, el más peligroso de los seis: tratar el Framework Canvas completado como un documento de uso único, archivado formalmente y efectivamente olvidado después de su creación inicial, en lugar de tratarlo como un artefacto vivo que se revisa de forma activa en cada punto de revisión motivado calendarizado a lo largo del proyecto. La señal de alerta correspondiente: ningún miembro del equipo recuerda con precisión dónde está guardado el canvas original apenas tres meses después de haberlo elaborado con cuidado.

EJERCICIO 22 — AUDITORÍA DE TU ÚLTIMO FRAMEWORK CANVAS

1. Revisa el último canvas o documento de planeación que tu equipo haya producido.
2. Marca cuáles de los seis errores de este capítulo están presentes.
3. Corrige al menos dos antes de que el proyecto avance a la siguiente fase.

Aplícalo a tu proyecto

| “Framear mal no se ve como un error. Se ve como confianza excesiva.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“Una herramienta que solo usa una persona es una preferencia individual. Una herramienta que usa un equipo, sin que nadie tenga que pedirla, es cultura.”

De la teoría a la cultura: instalar el hábito de framear

“Una herramienta que solo usa una persona es una preferencia individual. Una herramienta que usa un equipo, sin que nadie tenga que pedirla, es cultura.”

Ningún framework, por bien diseñado que esté en el papel, sobrevive más allá del primer trimestre de aplicación si su continuidad depende exclusivamente de la memoria individual o de la disciplina personal de una sola persona defendiéndolo, junta tras junta, frente a un equipo que no ha internalizado genuinamente su valor. Este capítulo final de la Parte V aborda, con la seriedad que merece, la pregunta más difícil —y, al mismo tiempo, la más importante— de todo el libro: ¿cómo se convierte el framing, de una técnica aprendida en la lectura individual de un libro, en un hábito que la organización completa practica de forma consistente sin que nadie tenga que recordarlo activamente cada vez?

La respuesta, fundamentada en patrones observados de forma consistente en equipos que efectivamente lograron instalar esta disciplina de forma duradera, tiene tres componentes estructurales identificables. Primero, ritual explícito en lugar de voluntad individual: el Framework Canvas debe constituirse como un requisito formal de entrada dentro del proceso de aprobación de proyectos de la organización, no como una buena práctica meramente opcional que depende, en última instancia, de quién específicamente esté a cargo del proyecto en cuestión. Si un proyecto propuesto no cuenta con un canvas completo y debidamente documentado, ese proyecto no debería avanzar a la cola formal de priorización organizacional. La fricción estructural incorporada al proceso resulta, de forma consistente, considerablemente más confiable a lo largo del tiempo que la buena intención individual, por genuina que esta sea en el momento inicial.

Segundo, lenguaje compartido y consistentemente reforzado: los

términos técnicos desarrollados a lo largo de este libro —fluent volátil, cierre por explicación, punto de revisión motivado, ramificación, framing por default— deben incorporarse de forma efectiva al vocabulario cotidiano y compartido del equipo completo, en lugar de permanecer como jerga especializada conocida únicamente por la persona que efectivamente leyó este libro completo. La forma más rápida y efectiva de lograr esta incorporación no es, según la experiencia acumulada, una única sesión de capacitación formal, sino el uso repetido, natural y consistente de estos mismos términos dentro de las juntas de seguimiento regulares del equipo, hasta que se conviertan, de forma orgánica, en parte genuina del idioma compartido y cotidiano del equipo completo.

Tercero, y quizás el componente más contraintuitivo de los tres: celebrar de forma explícita la detección temprana de un supuesto roto, no únicamente la ejecución exitosa de un proyecto conforme al plan original. Cuando un punto de revisión motivado detecta, con antelación suficiente, que un supuesto crítico efectivamente se rompió, y el equipo ajusta el plan correspondiente antes de que el problema escale a una crisis mayor, ese evento específico merece reconocimiento explícito dentro de las retrospectivas formales del equipo, con exactamente el mismo peso relativo que se otorga habitualmente a un lanzamiento exitoso conforme al plan. Si la cultura de tu organización únicamente celebra, de forma sistemática, los proyectos que "salieron bien" al final, sin distinguir de forma explícita entre buena suerte circunstancial y buen framing deliberado, estás entrenando, de forma involuntaria pero efectiva, a tu equipo para preferir sistemáticamente la apariencia de confianza sobre la disciplina genuina y verificable.

EJERCICIO 23 — DISEÑA TU PLAN DE INSTALACIÓN CULTURAL

1. Identifica dónde en tu proceso actual de aprobación de proyectos podrías insertar el Framework Canvas como requisito de entrada.
2. Elige tres términos de este libro que quieras instalar en el vocabulario cotidiano de tu equipo en los próximos 90 días.
3. Define cómo vas a reconocer, en tu próxima retrospectiva, un caso de detección temprana de un supuesto roto.

Aplícalo a tu proyecto

“Una herramienta que solo usa una persona es una preferencia individual. Una herramienta que usa un equipo, sin que nadie tenga que pedirla, es cultura.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

“No vas a encontrar el framing correcto esperando a tener suficientes datos. Vas a construirlo, con criterio, y vas a revisarlo cuando el mundo te avise que cambió.”

Cierre: el framing es una decisión, no un descubrimiento

“No vas a encontrar el framing correcto esperando a tener suficientes datos. Vas a construirlo, con criterio, y vas a revisarlo cuando el mundo te avise que cambió.”

Empezamos este libro con una idea deliberadamente incómoda: la parte más difícil de decidir dentro de un mundo que cambia de forma constante no es calcular, con la mayor precisión posible, qué va a pasar en el futuro. Es decidir, de forma eficiente y honesta, qué vas a asumir que se mantiene igual mientras tanto. Más de veintiséis años de investigación intensa dentro de la inteligencia artificial simbólica no lograron resolver de forma completa el frame problem en su formulación más general y formal (Morgenstern, 1996). Pero en el intento sostenido de resolverlo, esa investigación construyó un conjunto de principios que, traducidos con cuidado al lenguaje del negocio, de los datos y de la inteligencia artificial aplicada, resultan directamente utilizables desde mañana mismo, sin necesidad de dominar ninguna notación lógica formal.

El mensaje final de este libro no es que ahora dispongas de un método que elimina por completo la incertidumbre estructural inherente a cualquier decisión relevante. Ningún framework la elimina de forma genuina, y cualquier propuesta que te prometa exactamente eso —incluyendo, si se malinterpreta de forma superficial, este mismo libro— está, en algún sentido relevante, vendiendo una ilusión que no se sostiene bajo escrutinio riguroso. El mensaje real es considerablemente más modesto, y al mismo tiempo considerablemente más útil en la práctica cotidiana: ahora cuentas con un vocabulario preciso para nombrar con exactitud por qué un plan determinado falló, un catálogo razonablemente completo de herramientas concretas para reducir la probabilidad de que vuelva a fallar exactamente de la misma manera

estructural, y una disciplina operativa concreta —el Framework Canvas, la estructura de facilitación de sesiones de equipo, los puntos de revisión motivados— para instalar de forma efectiva esta mejora dentro de tu equipo completo, no únicamente dentro de tu propia cabeza individual.

El frame problem, estudiado con el detalle que le hemos dedicado a lo largo de este libro, enseña, en el fondo, una lección genuina de humildad operativa: no es posible escribir suficientes reglas explícitas para cubrir cada caso particular que pueda presentarse, y tampoco deberías intentarlo de forma exhaustiva, porque ese camino conduce, de forma predecible, de vuelta a la explosión combinatoria descrita ya desde el Capítulo 2 de este libro. Lo que sí puedes hacer, de forma realista y sostenible en el tiempo, es ser deliberado y explícito sobre qué estás asumiendo en cada decisión relevante, transparente sobre qué específicamente no has verificado todavía con evidencia directa, y disciplinado sobre cuándo y cómo revisar esos supuestos de forma proactiva, antes de que el mundo te obligue a revisarlos de forma reactiva, en medio de una crisis ya en desarrollo.

Framear bien no constituye un talento innato con el que se nace o no se nace. Es, de forma demostrable a través de los casos y ejemplos desarrollados a lo largo de este libro, una práctica que se instala de forma progresiva, capítulo a capítulo, sesión de trabajo a sesión de trabajo, canvas a canvas. La próxima decisión de cierta relevancia que vayas a tomar —hoy mismo, esta semana, en tu próximo proyecto de datos o de negocio— constituye una oportunidad concreta para practicarla de forma deliberada. El framing correcto no está esperando pasivamente a que lo descubras en algún momento futuro con suficiente información acumulada. Está esperando, de forma activa, a que lo decidas ahora, con el criterio disponible hoy, y a que lo revises de forma disciplinada cuando la evidencia disponible te indique que es momento de hacerlo.

EJERCICIO FINAL — TU COMPROMISO DE FRAMING

1. Elige una decisión concreta que tomarás en los próximos 30 días.
2. Completa un Framework Canvas completo antes de tomarla.
3. Comparte este libro, o al menos el canvas resultante, con una persona de tu equipo, y pídele que te ayude a sostener el hábito.

Aplicálo a tu proyecto

“No vas a encontrar el framing correcto esperando a tener suficientes datos. Vas a construirlo, con criterio, y vas a revisarlo cuando el mundo te avise que cambió.”

¿QUÉ IDEA DE ESTE CAPÍTULO APLICA DIRECTAMENTE A UN PROYECTO TUYO HOY?

¿QUÉ SUPUESTO PROPIO IDENTIFICASTE AL LEER ESTE CAPÍTULO?

PRÓXIMO PASO CONCRETO (ESTA SEMANA)

RESPONSABLE Y FECHA

CIERRE DE PARTE V

Antes de continuar

¿Qué necesitas para que framear se vuelva un hábito de equipo, no una preferencia personal?

TU RESPUESTA

UNA ACCIÓN CONCRETA PARA ESTA SEMANA

Apéndice A

Herramientas del Toolkit

FICHA 1

Framework Canvas

Completa esta página, en equipo, antes de comenzar cualquier proyecto significativo. Ver Capítulo 25.

1. SITUACIÓN — ¿QUÉ FOTOGRAFÍA DEL MUNDO ESTAMOS TOMANDO?

2. FLUENTS VOLÁTILES — QUÉ CAMBIA CON CASI CADA ACCIÓN RELEVANTE

3. FLUENTS ESTABLES ASUMIDOS — QUÉ ASUMIMOS POR DEFAULT (Y QUÉ LO ROMPERÍA)

4. CAUSAS LEGÍTIMAS DE CAMBIO DEL DATO MÁS CRÍTICO

5. RIESGO DE CONCURRENCIA Y PUNTOS DE REVISIÓN MOTIVADOS

6. RAMIFICACIONES ESPERADAS Y RESPONSABLES

HTDQ 2.0 — Pregunta framed

| Ver Capítulo 26.

HÉROE — SISTEMA O PROCESO QUE EL PROYECTO BUSCA FORTALECER

TESORO — FLUENT QUE QUEREMOS MEJORAR, DE FORMA MEDIBLE

DRAGÓN — SUPUESTO DE PERSISTENCIA MÁS FRÁGIL DEL PROYECTO

PREGUNTA FRAMED — COMBINA LAS TRES PIEZAS + QUÉ VIGILAR / QUÉ ASUMIR

Checklist de framing pre-lanzamiento

Revisa esta lista antes de que cualquier proyecto de datos, producto o IA pase a producción.

- ☐ ¿Marcamos cada supuesto como "verificado" o "asumido por default"? (Cap. 12)
- ☐ ¿El Framework Canvas fue completado en equipo, no en solitario? (Cap. 27)
- ☐ ¿El canvas cabe en una página, sin enumerar lo obvio? (Cap. 7 y 28)
- ☐ ¿Existen puntos de revisión motivados explícitos en el calendario del proyecto? (Cap. 22)
- ☐ ¿Cada riesgo y ramificación tiene un responsable nombrado? (Cap. 28)
- ☐ ¿Definimos la lista corta y exhaustiva de causas legítimas de cambio del dato crítico? (Cap. 14)
- ☐ ¿Identificamos posibles "diamantes de Nixon" entre áreas que comparten esta decisión? (Cap. 10)
- ☐ ¿Ordenamos las hipótesis de anomalías por número de eventos no motivados? (Cap. 16)
- ☐ ¿El canvas quedó guardado en un lugar donde el equipo lo revisará, no solo archivado? (Cap. 29)

FICHA 4

Bitácora de punto de revisión motivado

Una ficha por cada punto de revisión definido en tu Framework Canvas. Ver Capítulo 22.

SUPUESTO QUE ESTAMOS VERIFICANDO

FECHA DE REVISIÓN Y RESPONSABLE

¿SIGUE VIGENTE EL SUPUESTO? EVIDENCIA.

RUTA DE AJUSTE (SI EL SUPUESTO YA NO ES VÁLIDO)

FICHA 5

Registro de causas legítimas de cambio

| Para el dato crítico de tu proyecto. Ver Capítulo 14.

DATO CRÍTICO

CAUSA LEGÍTIMA 1

CAUSA LEGÍTIMA 2

CAUSA LEGÍTIMA 3

¿QUIÉN Y CÓMO DETECTA UN CAMBIO FUERA DE ESTA LISTA?

Persistencia por default

¿Qué propiedades de tu proyecto asumes que persisten? ¿Qué anormalidad las rompería?

APLICACIÓN A TU PROYECTO ACTUAL

RIESGO SI NO SE APLICA

RESPONSABLE DE MANTENER ESTA DISCIPLINA

Cierre por explicación

¿Cuál es la lista corta y exhaustiva de causas legítimas de cambio de tu dato más crítico?

APLICACIÓN A TU PROYECTO ACTUAL

RIESGO SI NO SE APLICA

RESPONSABLE DE MANTENER ESTA DISCIPLINA

FICHA DE HERRAMIENTA

STRIPS (tres listas)

Define precondiciones, lista de adición y lista de eliminación para tu proceso.

APLICACIÓN A TU PROYECTO ACTUAL

RIESGO SI NO SE APLICA

RESPONSABLE DE MANTENER ESTA DISCIPLINA

Acción motivada

Ante una anomalía, ¿qué hipótesis requiere menos eventos sin causa conocida?

APLICACIÓN A TU PROYECTO ACTUAL

RIESGO SI NO SE APLICA

RESPONSABLE DE MANTENER ESTA DISCIPLINA

Circunscripción

¿Cuál es la regla general para tu población, y cuáles son las excepciones documentadas?

APLICACIÓN A TU PROYECTO ACTUAL

RIESGO SI NO SE APLICA

RESPONSABLE DE MANTENER ESTA DISCIPLINA

FICHA DE FACILITACIÓN

Agenda de sesión de framing (90 min)

| Ver Capítulo 27.

MINUTOS 0-15 · INDIVIDUAL Y EN SILENCIO — PARTICIPANTES Y NOTAS

MINUTOS 15-40 · COMPARACIÓN DE DIVERGENCIAS DETECTADAS

MINUTOS 40-70 · CONSTRUCCIÓN CONJUNTA (CAUSAS, CONCURRENCIA, RAMIFICACIONES)

MINUTOS 70-90 · DUEÑOS ASIGNADOS POR RIESGO

FICHA DE DIAGNÓSTICO

Post-mortem de un supuesto roto

Úsala cuando un supuesto falle — sin buscar culpables, buscando el patrón. Ver Capítulo 23.

¿QUÉ SUPUESTO SE ROMPIÓ Y CUÁNDO LO NOTAMOS?

¿ESTABA DOCUMENTADO COMO "VERIFICADO" O "ASUMIDO POR DEFAULT"?

¿QUÉ HERRAMIENTA DEL TOOLKIT LO HABRÍA PREVENIDO?

AJUSTE AL PROCESO PARA LA PRÓXIMA VEZ

Glosario

Frame problem (problema del framing)

El problema de determinar, de forma eficiente y sin enumerar reglas exhaustivas, qué permanece igual en un mundo que cambia con cada acción (McCarthy & Hayes, 1969).

Fluent

Cualquier propiedad del mundo, del negocio o de un sistema de datos que puede cambiar de valor a lo largo del tiempo.

Situación (situation)

Una fotografía del estado del mundo, del negocio o de un sistema en un instante específico, dentro de la situation calculus.

Framing

El proceso activo de decidir qué parte del mundo se va a considerar y cuál se va a dejar fuera del alcance de una decisión o sistema.

Persistencia por default

El principio de asumir que una propiedad se mantiene igual después de una acción, salvo que exista una anormalidad explícita que la afecte (McCarthy, 1980).

Cierre por explicación (explanation closure)

El principio de que una propiedad solo puede cambiar si ocurre una de sus causas legítimas conocidas (Davis, 1990; Schubert, 1990).

Concurrencia

La ocurrencia simultánea de múltiples acciones, muchas de ellas fuera del control o del conocimiento de quien toma la decisión.

Ramificación (ramification problem)

Un efecto derivado o en cascada de una acción, causado por una dependencia estructural preexistente, no por la acción de forma directa (Finger, 1988).

Acción motivada (motivated action)

Un evento para el cual existe una razón o causa explícita dentro del modelo; Motivated Action Theory prefiere las explicaciones con menos eventos sin causa conocida (Stein & Morgenstern, 1994).

Circunscripción (circumscription)

Una técnica de razonamiento que minimiza los casos considerados "anormales", asumiendo comportamiento general salvo evidencia específica de excepción (McCarthy, 1980).

Yale Shooting Problem

Contraejemplo formulado por Hanks y McDermott (1986) que mostró que la lógica de persistencia de McCarthy permitía conclusiones formalmente válidas pero intuitivamente absurdas.

Punto de revisión motivado

Un momento explícito, planeado dentro de un proyecto, dedicado a verificar si un supuesto de persistencia específico sigue siendo válido.

Framework Canvas

La plantilla de una página propuesta en este libro para hacer explícitos los supuestos de framing de cualquier proyecto antes de comenzar a construir.

Diamante de Nixon organizacional

Situación en la que dos áreas o personas aplican defaults razonables pero incompatibles a la misma decisión (Reiter & Criscuolo, 1981).

Referencias

Formato APA (7.^a edición).

- Davis, E. (1990). Representations of commonsense knowledge. Morgan Kaufmann.
- Enders, A., & Chevallier, A. (2022). Solvable: A simple solution to complex problems. Financial Times Publishing.
- Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3-4), 189-208.
- Finger, J. J. (1988). Exploiting constraints in design synthesis (Technical Report No. STAN-CS-88-1204). Stanford University.
- Gelfond, M., Lifschitz, V., & Rabinov, A. (1991). What are the limitations of the situation calculus? En R. Boyer (Ed.), *Essays in honor of Woody Bledsoe*. Kluwer Academic Publishers.
- Hanks, S., & McDermott, D. (1986). Default reasoning, nonmonotonic logic, and the frame problem. *Proceedings of the Fifth National Conference on Artificial Intelligence (AAAI-86)*, 328-333.
- McCarthy, J. (1980). Circumscription — A form of non-monotonic reasoning. *Artificial Intelligence*, 13(1-2), 27-39.
- McCarthy, J. (1986). Applications of circumscription to formalizing common-sense knowledge. *Artificial Intelligence*, 28(1), 86-116.
- McCarthy, J., & Hayes, P. J. (1969). Some philosophical problems from the standpoint of artificial intelligence. En B. Meltzer & D. Michie (Eds.), *Machine Intelligence* (Vol. 4, pp. 463-502). Edinburgh University Press.
- McDermott, D. (1982). A temporal logic for reasoning about processes and plans. *Cognitive Science*, 6(2), 101-155.
- Morgenstern, L., & Stein, L. A. (1988). Why things go wrong: A formal theory of causal reasoning. *Proceedings of the Seventh National Conference on Artificial Intelligence (AAAI-88)*, 518-523.
- Morgenstern, L. (1996). The problem with solutions to the frame problem. En K. M. Ford & Z. W. Pylyshyn (Eds.), *The robot's dilemma revisited: The frame problem in artificial intelligence* (pp. 99-133). Ablex Publishing.
- Pednault, E. P. D. (1991). Generalizing nonlinear plans to handle complex goals and actions with context-dependent effects. *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence (IJCAI-91)*, 240-245.

- Penberthy, J. S. (1993). Planning with continuous change (Technical Report No. 93-12-01). University of Washington, Department of Computer Science and Engineering.
- Reiter, R. (1991). The frame problem in the situation calculus: A simple solution (sometimes) and a completeness result for goal regression. En V. Lifschitz (Ed.), *Artificial intelligence and mathematical theory of computation: Papers in honor of John McCarthy* (pp. 359-380). Academic Press.
- Reiter, R., & Criscuolo, G. (1981). On interacting defaults. *Proceedings of the Seventh International Joint Conference on Artificial Intelligence (IJCAI-81)*, 270-276.
- Schubert, L. K. (1990). Monotonic solution of the frame problem in the situation calculus: An efficient method for worlds with fully specified actions. En H. E. Kyburg, R. P. Loui, & G. N. Carlson (Eds.), *Knowledge representation and defeasible reasoning* (pp. 23-67). Kluwer Academic Publishers.
- Shoham, Y. (1988). *Reasoning about change: Time and causation from the standpoint of artificial intelligence*. MIT Press.
- Stein, L. A., & Morgenstern, L. (1994). Motivated action theory: A formal theory of causal reasoning. *Artificial Intelligence*, 71(1), 1-42.

Sobre el autor

Jesús Mario González Siller es Arquitecto de Soluciones y Product Owner de Datos & Analítica en FEMSA, donde lidera iniciativas de plataformas de datos, inteligencia artificial y analítica retail para operaciones en múltiples países de América Latina y Europa.

Escribe regularmente sobre ingeniería de datos, arquitectura, inteligencia artificial aplicada y retail en jgonzalez.app, y comparte plantillas y recursos prácticos en The Toolkit, su colección de herramientas para profesionales de datos y negocio.

Preguntas frecuentes

¿Necesito saber lógica o inteligencia artificial académica para usar este libro?

No. El libro toma prestado el vocabulario del frame problem (problema del framing), pero cada herramienta se explica y se aplica en términos de negocio, datos y procesos, sin notación formal.

¿Para qué roles es más útil este libro?

Para líderes y practicantes de datos, analítica, producto e inteligencia artificial que diseñan sistemas, modelos o procesos que deben seguir siendo válidos cuando el negocio cambia.

¿Puedo usar el Framework Canvas con un equipo no técnico?

Sí. Las seis secciones del canvas están diseñadas para completarse en lenguaje de negocio; ningún término requiere formación técnica previa.

¿Cómo se relaciona este libro con otros recursos de The Toolkit?

Es el manual de fundamentos de decisión que sostiene a los demás recursos prácticos de la colección — ver la última sección de este libro.

Sigue construyendo tu Toolkit

Este libro es el primero de una colección de recursos prácticos que comparto en jgonzalez.app/the-toolkit: plantillas, kits y manuales de datos, analítica e inteligencia artificial, pensados para aplicarse directo en proyectos reales, sin reinventar la rueda cada vez.

Si este libro te fue útil, en The Toolkit encontrarás recursos complementarios: desde cursos de modelado de datos hasta kits de SQL para retail y manuales de prompting profesional — todos con el mismo enfoque de este libro: teoría mínima necesaria, ejemplos reales, y una plantilla que puedas usar hoy.

The Toolkit — jgonzalez.app/the-toolkit